
LEGO: LLM-based Evaluation and Guided Optimization for Adaptive Algorithm Design

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 To efficiently solve optimization problems, a wide variety of algorithms have
2 been developed, each designed to perform well under specific problem structures
3 or domains. However, due to the no-free-lunch theorem, no single algorithm
4 consistently outperforms others across all instances. This raises a fundamental
5 question: **how can we automatically construct algorithms tailored to the given**
6 **problem?** Inspired by LEGO-style modularity, we propose LEGO, a general
7 framework for adaptive algorithm design via pipeline component selection. Using
8 Mixed Integer Linear Programming (MILP) as a prototype, we construct solving
9 pipelines by selecting and configuring components and hyperparameter within
10 the Predict-and-Search paradigm. To ensure adaptability across varying data
11 scales, LEGO can self-adaptively generate synthetic datasets of different sizes,
12 enabling robust configuration even with limited data. It leverages large language
13 models (LLMs) to evaluate and guidedly optimize candidate configurations, using a
14 hybrid metric that combines classical performance indicators with LLM-informed
15 assessments. High-quality pipelines are selected through hypervolume-based
16 ranking and further refined via performance transfer on synthetic data to improve
17 scalability. Experiments on four benchmark MILP tasks demonstrate that the
18 proposed evaluation framework effectively identifies high-performing strategies and
19 hyperparameter configurations, leading to algorithms that are both more efficient
20 and more effective, highlighting LEGO as a generalized framework for component
21 and hyperparameter selection in MILP solving frameworks, with potential for
22 extension to broader algorithm design.

23 1 Introduction

24 Designing efficient algorithms for solving hard optimization problems is a core challenge across
25 scientific and engineering domains. Over the years, a wide variety of algorithms have been proposed,
26 each tailored to specific problem structures or domains [1, 2, 3]. However, the *no-free-lunch theorem*
27 implies that no single algorithm can consistently outperform all others across the full spectrum of
28 problem instances [4]. This raises a fundamental question: **how can we automatically construct**
29 **algorithms that are tailored to a given problem?**

30 In this paper, we address this question by proposing **LEGO**, a general framework for adaptive
31 algorithm design via modular pipeline construction and optimization. Inspired by the flexibility of
32 LEGO-style block assembly, LEGO treats algorithm design as a combinatorial construction problem—selecting and configuring components and hyperparameters within a parameterized framework.
33 While the core ideas are broadly applicable, we instantiate and validate LEGO in the domain of
34 *Mixed Integer Linear Programming* (MILP), a widely adopted modeling paradigm for combinatorial
35 optimization tasks such as routing [5], scheduling [6], and supply chain optimization [7].
36

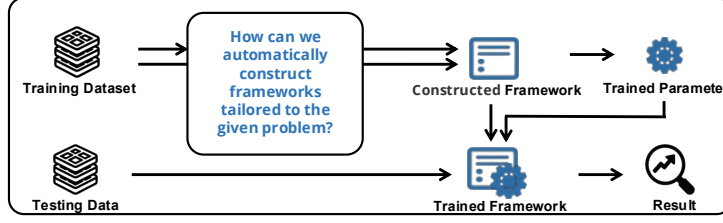


Figure 1: Core research question: Given a problem distribution, how can we construct a solving framework tailored to it? LEGO automatically assembles and tunes a solving pipeline based on training data, and transfers it to larger test instances.

To ground our investigation, we build upon the *Predict-and-Search* (P&S) paradigm [8, 9], which structures MILP solvers into modular stages such as graph embedding, neural prediction, solution repair, and heuristic search. Existing P&S-based frameworks often commit to fixed designs and fixed-size data configurations, limiting their adaptability. In contrast, LEGO decomposes the P&S architecture into interchangeable components and systematically explores the space of configurations, enabling automated construction of solving pipelines tailored to the task at hand.

A key challenge in this process is how to evaluate and select from a large pool of candidate solver configurations. To this end, LEGO introduces an **LLM-enhanced performance evaluation** mechanism, which combines classical indicators (e.g., objective gap, runtime) with LLM-informed metrics to assess solving behavior in a more holistic and task-aware way. These evaluations are then aggregated using a **hypervolume-based component selection** strategy to identify high-quality solver pipelines under multiple objectives. Furthermore, LEGO supports generalization across problem scales. Even when only small-scale training data are available, LEGO leverages a **scalable data generation** module to synthesize large-scale instances with similar structure, enabling robust **size-transferable parameter tuning**. This ensures that the selected pipelines can be effectively adapted to larger and more complex problems, even under limited training dataset. A demo of LEGO is available at <https://anonymous.4open.science/r/LEGO-B36D>, and the full codebase will be released after the review process.

Our contributions are summarized as follows:

- **Scale-aware adaptive framework construction.** LEGO introduces a flexible mechanism to generate synthetic instances of arbitrary scale with high structural similarity, enabling algorithm configuration even under data-scarce or size-mismatched scenarios.
- **A unified LLM-guided optimization framework.** We propose three key techniques to guide the construction of solving pipelines: (1) **LLM-enhanced Performance Evaluation**, which combines classical metrics with LLM-informed assessments; (2) **Hypervolume-guided Component Selection**, which enables robust multi-objective ranking; and (3) **Size-transferable Parameter Tuning**, which refines pipeline performance across scales.
- **Empirical validation on MILP benchmarks.** We validate LEGO on four widely used MILP benchmarks (MIS, MVC, SC, MKS). Results show that LEGO consistently discovers high-performing solver frameworks, outperforming both classical solvers (e.g., Gurobi, SCIP) and ML-based baselines (e.g., Light-MILPopt).

2 Related Work

2.1 Automatically Algorithm Design

Automatically Algorithm Design (AAD) seeks to construct or adapt algorithms to specific problem distributions, fundamentally motivated by the "no-free-lunch" theorem, which states that no single algorithm excels on all problems. Classical AAD paradigms include *Algorithm Configuration* (AC) [10, 11], which optimizes hyperparameters for a fixed algorithm (e.g., ParamILS, LEAPSAND-BOUNDS), and *Algorithm Selection* (AS) [12], which chooses the most suitable algorithm from a predefined portfolio based on instance features. These approaches are often formalized under

76 the *Meta-Black-Box Optimization* (MetaBBO) framework [13], where meta-level learning drives
 77 algorithmic adaptation.

78 While effective in many settings, these methods typically treat algorithms as atomic units, limiting
 79 flexibility and reusability [14]. Recent trends move toward more granular, component-based AAD
 80 [15], where algorithms are assembled from interchangeable functional units. This modular design
 81 enables finer control and potentially richer adaptation. However, challenges remain: most existing
 82 systems struggle with generalization across problem scales [16], as configurations optimized on one
 83 size often degrade on others. Moreover, managing inter-component dependencies and avoiding
 84 performance bottlenecks in dynamic or large-scale settings remains a significant open problem.

85 2.2 Mixed Integer Linear Programs

86 Mixed Integer Linear Programs (MILPs) are a fundamental class of combinatorial optimization
 87 problems, defined by a linear objective function with linear constraints, where some variables are
 88 restricted to take integer values [17]. The general form of an MILP is given by:

$$\min_x c^T x \quad \text{s.t.} \quad Ax \leq b, \quad l \leq x \leq u, \quad x_i \in \mathbb{Z} \text{ for } i \in \mathbb{I}, \quad (1)$$

89 where $x \in \mathbb{R}^n$ denotes the decision variables, $c \in \mathbb{R}^n$ the objective coefficients, $A \in \mathbb{R}^{m \times n}$ and
 90 $b \in \mathbb{R}^m$ define the linear constraints, and $l, u \in \mathbb{R}^n$ represent variable bounds. The index set
 91 $\mathbb{I} \subseteq \{1, \dots, n\}$ indicates the subset of variables that must take integer values.

92 Solving MILPs is NP-hard in general [18], and exact methods such as branch-and-bound, branch-
 93 and-cut, and cutting plane techniques remain the backbone of modern solvers [17]. Despite the
 94 success of commercial tools like Gurobi and open-source solvers like SCIP, large-scale or real-time
 95 MILPs often remain computationally intractable. Recent efforts have explored hybrid approaches,
 96 combining classical methods with learning-based components to improve scalability and adaptability
 97 [19], giving rise to modular frameworks such as the *Predict-and-Search* paradigm.

98 2.3 Predict-and-Search

99 The Predict-and-Search (P&S) paradigm [8, 9] offers a flexible framework that integrates learning-
 100 based modules into traditional optimization solvers by structuring them into distinct stages—typically
 101 involving a prediction phase followed by a search phase. This modular decomposition facilitates the
 102 injection of data-driven components to tailor solver behavior. However, both the prediction and search
 103 stages admit a wide range of possible designs—e.g., different neural architectures, scoring heuristics,
 104 or branching rules—leading to a large combinatorial space of solver configurations. Moreover, the
 105 performance of a given configuration often varies significantly across problem scales or distributions.
 106 These factors highlight the need for automatic algorithm design methods that can adaptively select and
 107 compose solver components, while ensuring robustness and generalization across diverse instances.

108 3 Method

109 We propose **LEGO**, a general framework for adaptive algorithm design within a fixed problem
 110 domain. LEGO automatically constructs high-performance algorithms tailored to a specific class of
 111 optimization problems by assembling solving pipelines from modular components and optimizing
 112 them via LLM-guided evaluation and search. To support adaptation across instance scales, LEGO
 113 integrates a synthetic instance generator that enables effective pipeline tuning even with limited
 114 training data.

115 As shown in Figure 2, LEGO consists of two main modules. The upper module—**LLM-based**
 116 **Evaluation and Guided Optimization**—serves as the general optimization engine, combining
 117 hybrid metric evaluation with multi-objective search to guide pipeline construction. To demonstrate
 118 the framework’s effectiveness, we instantiate LEGO for *Mixed Integer Linear Programming* (MILP),
 119 building a **Component Library** based on the Predict-and-Search (P&S) paradigm. This library
 120 includes interchangeable components such as graph embeddings, neural predictors, repair heuristics,
 121 and search strategies. LEGO composes and tunes these components to construct scalable and adaptive
 122 MILP solvers.

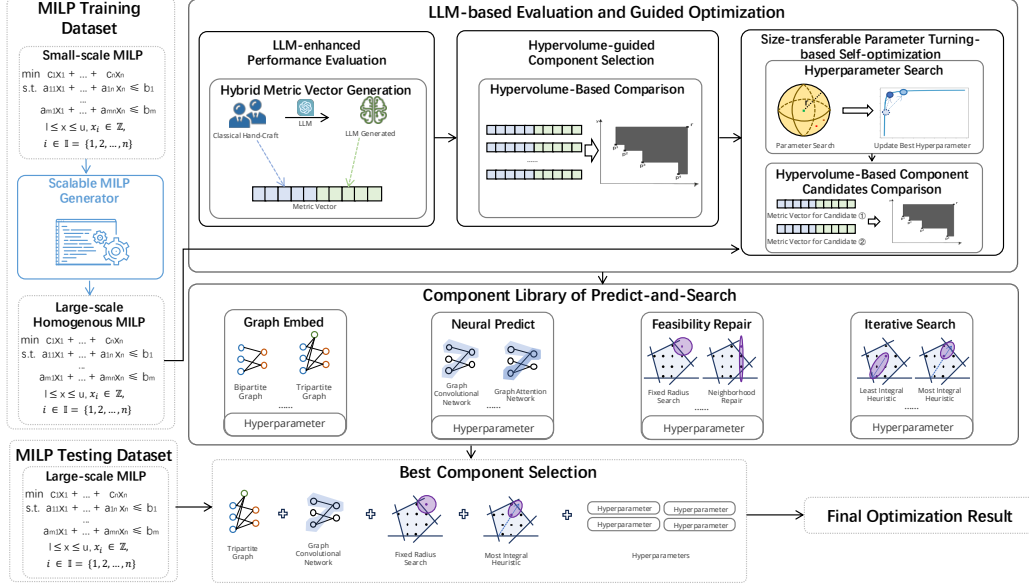


Figure 2: Overview of the proposed **LEGO** framework. Solving pipelines are optimized via **LLM-based Evaluation and Guided Optimization**, and assembled from the modular **Component Library** following the Predict-and-Search paradigm. Optional blue modules (e.g., the Scalable MILP generator) support adaptation to large-scale instances when training data is limited.

3.1 LLM-based Evaluation and Self-Optimization

To evaluate and optimize component combinations within LEGO, we design a unified framework that integrates multi-dimensional performance metrics, including both classical solver indicators and task-specific criteria automatically generated by large language models (LLMs). These *LLM-enhanced performance evaluations* enable context-aware, goal-aligned assessment of solver behavior. To compare diverse configurations, we adopt a *hypervolume-based selection* strategy that jointly considers multiple objectives. To reduce evaluation cost, candidate pipelines are first assessed on small-scale instances, then adapted to larger problems through *size-transferable parameter tuning*, enabling efficient self-optimization with generalization across instance scales.

3.1.1 LLM-enhanced Performance Evaluation

To comprehensively evaluate pipelines constructed from the LEGO component library, we adopt a hybrid metric framework that combines classical human-designed indicators with task-aware criteria generated by large language models (LLMs). Specifically, we define a set of seven evaluation metrics that capture both solution quality and search dynamics over time.

We first construct four classical metrics based on oracle-level performance. For each training instance, we run a strong solver for a long time to obtain a high-quality upper bound x^* , and evaluate pipelines by: (1) the initial solution gap to x^* , (2) the final solution gap to x^* , (3) the *efficiency rate*, defined as the fraction of instances with final gap $\leq 10\%$, and (4) the time to reach the first valid solution (gap $\leq 10\%$) and the first high-quality solution (gap $\leq 1\%$). To complement these, we employ GPT-4o [20] to generate candidate evaluation criteria from solver trajectories. After filtering and human validation, we retain three additional metrics: (5) the solution gap at 20% of the time budget, (6) the gap at 60% of the time budget, and (7) the time-integrated solution gap over the full horizon (i.e., area under the gap-time curve). Details on prompt design, filtering strategy, and implementation are provided in the appendix.

The resulting 7-dimensional evaluation vector offers a fine-grained, multi-perspective assessment of solver performance, combining expert insight with LLM-generated domain knowledge to guide downstream optimization and selection.

3.1.2 Hypervolume-guided Component Selection

To efficiently select promising solver pipelines from the large combinatorial space of LEGO component configurations, we first evaluate all candidates on small-scale training instances using the *LLM-enhanced Performance Evaluation* described above, resulting in a 7-dimensional performance vector for each.

Due to the large number of configurations, we apply non-dominated sorting to identify the Pareto frontier S , which contains all configurations that are not strictly outperformed across all evaluation dimensions. This step filters out clearly suboptimal pipelines and retains those that offer distinct trade-offs in performance. However, the frontier S may still contain many similar or marginally different candidates. To further rank configurations within S , we compute the *hypervolume contribution* of each configuration $d \in S$:

$$\Delta V(d) = V(S) - V(S \setminus \{d\}),$$

where $V(\cdot)$ denotes the hypervolume with respect to a fixed reference point. A larger $\Delta V(d)$ indicates that configuration d contributes uniquely to the performance diversity of the Pareto set.

We rank all candidates in S by their hypervolume contribution and select the top- K configurations as solver pipelines for further tuning and deployment. This approach ensures efficient and diverse component selection, while keeping evaluation cost low by operating on small-scale instances.

3.1.3 Size-transferable Parameter Tuning

Since component selection is performed on small-scale training instances, we introduce a *Size-transferable Parameter Tuning* stage to ensure the resulting solver pipelines generalize effectively to larger-scale problems. This step jointly finalizes both component choices and their associated hyperparameters for deployment.

If the training dataset includes instances of varying sizes—especially those comparable to the test-time scale—we directly perform parameter tuning on the larger training instances. In the more common case where only small-scale data is available, we leverage a controllable instance generator based on MILP-retrieval [21], which can synthesize structurally similar problems with adjustable scale, difficulty, and similarity. This allows LEGO to adaptively tune solvers under any data regime.

In this stage, we apply Bayesian Optimization to search optimal hyperparameters for each of the top- K candidate configurations. The tuned candidates are then evaluated using the *LLM-enhanced Performance Evaluation* described earlier, and ranked using *Hypervolume-Based Component Candidates Comparison*. The final output is the best-performing configuration and its hyperparameters, optimized for both effectiveness and scalability.

3.2 Component Library

Building on the LLM-based evaluation and self-optimization framework introduced above, we apply LEGO to the important application domain of mixed-integer linear programming (MILP). Concretely, we instantiate LEGO within the widely adopted Predict-and-Search (P&S) paradigm, a dominant approach in learning-based MILP solving. Under this paradigm, LEGO decomposes the solver pipeline into four functional stages: *Graph Embed*, *Neural Predict*, *Feasibility Repair*, and *Iterative Search*. Each stage defines a modular interface with multiple candidate implementations, forming the LEGO Component Library. By systematically selecting and composing components across these stages, LEGO can generate diverse, adaptive solving frameworks tailored to different problem distributions and instance scales. A complete description of all components and their pseudocode is provided in the appendix.

3.2.1 Graph Embed

Directly feeding the raw algebraic form of a MILP problem into a neural model may obscure key structural invariances, such as row and column permutations that preserve problem equivalence. To retain these desirable symmetries, graph-based representations are commonly adopted, as they are inherently invariant to permutations of node order and layout. The role of the Graph Embed module is to transform a given MILP instance into a lossless graph representation that preserves its combinatorial and constraint structure.

In LEGO, we support several widely used encodings to capture the structure of MILPs: the *bipartite graph* representation [22], which connects variables and constraints as two disjoint node types; the *tripartite graph* representation [23], which further separates objective coefficients as a third node type; and two enhanced variants designed for foldable MILP instances: *bipartite with random feature strategy* and *tripartite with random feature strategy*, which inject randomized node features [24] to improve representation diversity in structurally repetitive problems called "foldable" problems. Each encoding defines a distinct component in the Graph Embed stage of our Component Library.

3.2.2 Neural Predict

Given the graph representation of a MILP instance, the Neural Predict module applies graph neural networks (GNNs) to learn mappings from problem structure to solution space. During training, a GNN is trained to predict optimal or near-optimal solutions based on the graph-structured input of MILP instances. At inference time, the network generalizes to unseen problems and provides predictions that serve as initial candidates or guidance for downstream search. LEGO integrates two widely used GNN architectures in this stage: the *Graph Convolutional Network* (GCN) [25] and the *Graph Attention Network* (GAT) [26], both featuring semi-convolutional designs to capture local and contextual structure. Network depth (i.e., number of layers) is treated as a tunable hyperparameter to support flexible capacity control during optimization.

For problem settings where feasible regions are extremely small, fragmented, or hard to learn, neural networks may struggle to produce high-quality or even feasible predictions. To address this, LEGO also supports solver-based predictors using commercial solvers such as *Gurobi* [27] and *SCIP* [28]. These solvers can generate initial feasible solutions reliably, even when neural predictors fail or are uncertain. Although solver-based initializations may be suboptimal compared to learned predictions in many cases, they offer robustness in challenging problem domains. LEGO thus enables hybrid designs within the Neural Predict stage, where learning-based and solver-based components can be used individually or in combination, depending on the characteristics of the target problem.

3.2.3 Feasibility Repair

The predictions generated by the Neural Predict module are not guaranteed to satisfy all constraints of the original MILP problem. Directly using such infeasible solutions in downstream search often leads to inefficient or invalid trajectories. To address this, the Feasibility Repair module aims to transform potentially infeasible predictions into valid solutions that respect problem constraints.

LEGO integrates three complementary repair strategies. The first is the *adaptive radius search* [8], which defines a dynamic neighborhood around the predicted solution and invokes a solver to explore feasible candidates within this radius. This approach balances prediction guidance with combinatorial search flexibility. The second is the *adaptive threshold* method [29], which adjusts the prediction confidence threshold and delegates the unresolved portion of the solution to a lightweight solver. The third is the *neighborhood repair* strategy [30], which prunes the prediction set based on local constraint structures to reduce infeasibility. These strategies are implemented as interchangeable components in the repair stage. Detailed algorithmic descriptions can be found in the Appendix.

3.2.4 Iterative Search

Once a feasible solution is obtained, it can be further improved via iterative search, a widely adopted strategy in modern MILP solving frameworks. The core idea is to fix a subset of decision variables and iteratively refine a selected neighborhood using a solver. This process allows local exploration around promising solutions and can significantly enhance solution quality.

LEGO supports five distinct strategies in this stage. The first is classical *Large Neighborhood Search* (LNS) [31], which randomly selects a subset of variables for re-optimization. We further include *Adaptive Large Neighborhood Search* (ALNS) [32] with adaptive neighborhood size, which dynamically adjusts the scope of variables based on search feedback. In addition, we provide two heuristics based on the integrality of the relaxed LP solution: the *Least Integral Heuristic* (LIH) [33], which focuses on variables farthest from integral values, and the *Most Integral Heuristic* (MIH) [34], which prioritizes those closest to integrality. Finally, the *Adaptive Constraint Partition* (ACP) method [35] leverages variable correlations to construct meaningful subproblems for focused refinement.

Table 1: Comparison of objective value results with baseline approaches using the same execution time. An upward arrow ($\uparrow$) indicates that the result is better than the baseline. **Boldface** denotes the best result for each problem instance.

	SC ₁	SC ₂	MVC ₁	MVC ₂	MKS ₁	MKS ₂	MIS ₁	MIS ₂
Gurobi	24313.0	320036.5	27925.8	330816.4	34285.3	343707.2	-21966.7	-169223.2
SCIP	25317.5	919262.6	31256.7	490914.5	30616.0	1047136.9	-18687.9	-9125.2
Light-MILPopt	16528.1	164154.3	27548.1	278557.6	20589.2	208803.5	-22900.1	-228611.5
LEGO-Real (Ours)	16108.6$\uparrow$	160693.4 $\uparrow$	26675.4$\uparrow$	271168.7$\uparrow$	19957.7$\uparrow$	203306.6 $\uparrow$	-23085.9 $\uparrow$	-228792.6 $\uparrow$
LEGO-Gen (Ours)	16183.3 $\uparrow$	160647.8$\uparrow$	26709.9 $\uparrow$	273948.6 $\uparrow$	20063.3 $\uparrow$	203054.1$\uparrow$	-23204.0$\uparrow$	-230261.0$\uparrow$

Beyond local search, LEGO also allows the repaired prediction to guide commercial solvers directly. We integrate both *Gurobi* [27] and *SCIP* [28] as back-end solvers, enabling the predicted solution to serve as a warm-start or search bias. This dual-mode design—search-based refinement and solver-guided integration—makes this stage a flexible and powerful component in our framework.

4 Experiment

We conduct comprehensive experiments to evaluate the effectiveness, adaptability, and scalability of LEGO in solving mixed-integer linear programs (MILPs) through Predict-and-Search pipeline construction. The evaluation covers a diverse range of MILP problem settings, including both synthetic benchmarks and real-world instances. LEGO is compared against classical solvers and representative learning-based baselines, under standardized experimental protocols. The experimental settings are detailed in Section C.1. To ensure a fair and comprehensive comparison, we employ multiple evaluation metrics to assess the performance of all methods considered in this study. Specifically, we include: a detailed comparison of solution quality under the same running time (Section C.2), an evaluation of time efficiency under the same solution quality (Section 4.3), and a convergence analysis of the optimization process (Section 4.4).

4.1 Experimental Settings

Dataset. We consider four representative types of NP-hard MILP problems: Set Covering (SC) [36], Minimum Vertex Cover (MVC) [37], Maximum Independent Set (MIS) [38], and Mixed 0-1 Knapsack Set (MKS) [39]. For each problem type, we evaluate two representative scales: one with approximately 100K decision variables and constraints (e.g., MVC₁), and another with approximately 1M scale (e.g., MVC₂). All problems are formulated as minimization tasks. Detailed mathematical formulations, instance generation strategies, and exact instance sizes are provided in the appendix.

Baseline Approaches. We compare LEGO with three strong baselines: the state-of-the-art commercial solver *Gurobi* 12.0.1 [27], the academic open-source solver *SCIP* 9.2.1 [28], and the recent learning-based Predict-and-Search framework *Light-MILPopt* [26]. In addition, we evaluate two variants of LEGO. The first, *LEGO-Real*, uses access to large-scale training data for final size-transferable parameter tuning. The second, *LEGO-Gen*, assumes only small-scale training data is available and relies on our MILP-retrieval-based instance generator to synthesize large-scale instances for component selection and size-transferable parameter tuning. These two variants allow us to assess LEGO’s performance under both data-rich and data-limited scenarios.

Environment. All experiments are conducted on a server equipped with Intel Xeon Platinum 8375C CPUs (2.90GHz) and four NVIDIA TESLA V100 GPUs (32GB each). Detailed experimental settings and runtime configurations are provided in the appendix.

4.2 Comparisons of Solution Effectiveness

To compare the solution effectiveness of different solvers, we evaluate all methods under the same execution time budget across eight large-scale MILP benchmarks, covering four problem types (SC, MVC, MKS, MIS) and two problem scales (100K and 1M). All problems are formulated as minimization tasks, where smaller objective values indicate better solution quality. This setting aligns with real-world scenarios, where solvers are often required to deliver high-quality solutions within fixed time limits. As shown in Table 1, our method LEGO achieves clearly better or comparable results across all tasks.

Table 2: Comparison of execution times under the same target value. A greater-than symbol ($>$) indicates the inability to achieve the target objective function in some instances within the maximum running time. **Boldface** is used to denote the best results.

	SC ₁	SC ₂	MVC ₁	MVC ₂	MKS ₁	MKS ₂	MIS ₁	MIS ₂
Gurobi	>30000s	>30000s	>30000s	>30000s	>30000s	>30000s	>30000s	>30000s
SCIP	>30000s	>30000s	>30000s	>30000s	>30000s	>30000s	>30000s	>30000s
Light-MILPopt	597.4s	3477.0s	594.7s	3473.4s	3909.9s	7931.2s	590.8s	3468.9s
LEGO-Real (Ours)	228.4s ↑	1936.6s↑	103.8s↑	536.0s ↑	2264.6s↑	3285.1s ↑	155.9s ↑	3367.7s↑
LEGO-Gen (Ours)	473.0s↑	1542.1s ↑	76.3s ↑	786.4s↑	2038.3s ↑	4923.2s↑	225.0s↑	2438.9s ↑

Traditional solvers such as Gurobi and SCIP yield significantly worse objective values under the same time constraints. For instance, Gurobi performs poorly on MVC₂ and MIS₂, while SCIP shows large suboptimality on SC₂ and MKS₂, where its returned values are more than five times worse than those of LEGO. These results indicate that classical solvers struggle to make effective progress on extremely large MILPs within limited time, due to the exponential search space and lack of learned heuristics. This contrast highlights the need for scalable methods in large-instance settings.

Compared to the strong learning-based baseline Light-MILPopt, both LEGO variants achieve consistently better objective values on all benchmarks. LEGO-Real performs particularly well on tasks where the training and test distributions are closely aligned, such as MVC₁ and MKS₁. LEGO-Gen, on the other hand, often outperforms LEGO-Real on large-scale or more complex tasks like SC₂ and MIS₂, thanks to its broader training distribution via synthetic MILP generation. These improvements demonstrate the benefit of LEGO’s modular training strategy and its ability to generalize under different problem characteristics.

Overall, LEGO delivers strong and stable performance across diverse problem types and scales. Its hierarchical structure and component-wise learning enable it to adapt more effectively to large and complex MILPs than existing monolithic methods. The ability to outperform both traditional solvers and advanced learning-based baselines under the same time budget confirms LEGO’s effectiveness as a general and practical framework for high-quality large-scale MILP solving.

4.3 Comparisons of Solving Efficiency

We evaluate the time efficiency of different methods under the same target objective value, meaning that all solvers are required to reach the same solution quality, and we compare the time needed to do so. As shown in Table 2, the two traditional solvers, Gurobi and SCIP, fail to reach the target within the 30,000-second limit on all tested instances. This highlights the difficulty of scaling general-purpose MILP solvers to high-dimensional problems with hundreds of thousands or millions of variables and constraints.

In contrast, both LEGO-Real and LEGO-Gen are able to reach the target solutions in significantly less time than the learning-based baseline Light-MILPopt. For example, on MVC₁, LEGO-Real reduces execution time from 594.7s to 103.8s (**82.5%** faster), while LEGO-Gen further reduces it to just 76.3s (**87.2%** faster). On SC₂, LEGO-Gen achieves a **55.6%** speedup over Light-MILPopt, while LEGO-Real saves **44.3%** of execution time. On average, LEGO reaches the same target values 2–6× faster across different problem types and scales. These results demonstrate that LEGO achieves comparable or better efficiency than the best existing learning-based Predict-and-Search framework.

We also observe that LEGO-Real is generally faster than LEGO-Gen on smaller-scale problems, likely due to its access to real large-instance training data, which helps specialize the learned policies. However, on larger-scale or more diverse instances (e.g., SC₂, MIS₂), LEGO-Gen often performs better, showcasing its stronger generalization capability through synthetic instance retrieval and adaptive training. This complementary behavior between the two variants suggests that LEGO is effective both when real data is available and when generalization to unseen distributions is required.

We attribute LEGO’s superior efficiency to its flexible and adaptive design. By providing a rich set of modular components and a wide range of tunable hyperparameters, LEGO can adapt its optimization strategy to the structural characteristics of each specific MILP instance. This allows the framework to tailor its behavior more precisely, leading to faster convergence and better scalability. In contrast, Light-MILPopt follows a monolithic approach where a single learned policy is applied uniformly across all problems, limiting its ability to generalize or specialize to different tasks.

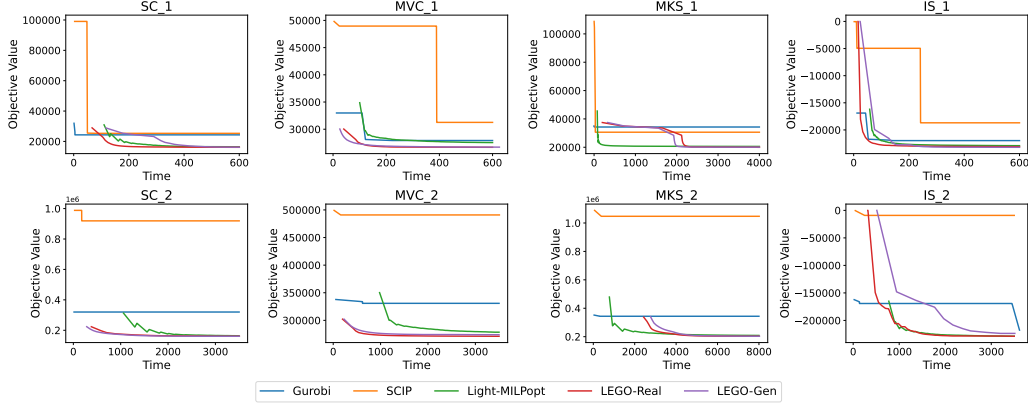


Figure 3: Time-objective convergence curves for all benchmark MILPs. Each subfigure shows the objective value (y-axis) over time (x-axis) for a specific MILP task.

LEGO’s compositional nature thus enables more problem-aware optimization and consistently high performance across diverse benchmarks.

4.4 Analysis of Convergence

We analyze the convergence behavior of all methods by visualizing how the objective value evolves over time across eight benchmark MILPs, as shown in Figure 3. Each curve reflects how efficiently a solver improves solution quality under a fixed time budget. The results show that both LEGO-Real and LEGO-Gen converge significantly faster than traditional solvers (Gurobi, SCIP) and the learning-based baseline Light-MILPopt, especially in early stages of optimization.

LEGO variants consistently achieve rapid descent in the objective value curve, reaching high-quality solutions within a few minutes on 100K-scale problems, and under a few hundred seconds on 1M-scale ones. In contrast, Gurobi and SCIP either plateau early or make slow progress, particularly on large-scale instances like SC₂ and MIS₂. Light-MILPopt shows moderate convergence but lags behind LEGO in nearly all tasks. These patterns demonstrate the effectiveness of LEGO’s structural decomposition and local decision policies, which enable faster and more focused optimization.

Moreover, LEGO-Gen often matches or even exceeds LEGO-Real in convergence speed on large-scale problems, such as MVC₂ and SC₂. This suggests that the synthetic training strategy and MILP retrieval mechanism empower LEGO-Gen with strong generalization and adaptation capabilities, even when real data is unavailable. Overall, the convergence curves highlight LEGO’s robustness, scalability, and practical efficiency in solving large MILPs.

5 Conclusion

We propose **LEGO**, a general and modular framework for adaptive algorithm construction through component selection and configuration. Inspired by LEGO-style modularity, the framework is designed to assemble high-performing solving pipelines tailored to specific problem instances. In this work, we instantiate LEGO within the Predict-and-Search paradigm for solving large-scale MILPs, where it selects and configures components and hyperparameters based on structural priors. To enhance adaptability, LEGO can generate synthetic training data at varying scales and leverage hybrid evaluation metrics, including LLM-informed assessments, to identify robust configurations even under limited training dataset.

Extensive experiments show that LEGO outperforms traditional solvers and strong learning-based baselines in both convergence speed and solution effectiveness. However, our current framework relies on the quality of the problem generator for training instance construction. In future work, we aim to further enhance the task-driven evaluation metrics to better align with practical objectives, and continuously expand the component library by integrating state-of-the-art modules, thereby improving LEGO’s adaptability and performance across diverse problem settings.

References

- [1] Stefan Bock, Stefan Bomsdorf, Nils Boysen, and Michael Schneider. A survey on the traveling salesman problem and its variants in a warehousing context. *European Journal of Operational Research*, 2025.
- [2] Meng-Yu Huang, Ling-Ying Huang, Yu-Xing Zhong, Hui-Wen Yang, Xiao-Meng Chen, Wei Huo, Jia-Zheng Wang, Fan Zhang, Bo Bai, and Ling Shi. Milp acceleration: A survey from perspectives of simplex initialization and learning-based branch and bound. *Journal of the operations research society of china*, 2025.
- [3] Phuong Le. A survey on combinatorial optimization. *arXiv preprint arXiv:2409.00075*, 2024.
- [4] Per Kristian Lehre and Shishen Lin. No free lunch theorem and black-box complexity analysis for adversarial optimisation. *Advances in Neural Information Processing Systems*, 2024.
- [5] Xiaojian Li and Kwan L Yeung. Traffic engineering in segment routing networks using milp. *IEEE Transactions on Network and Service Management*, 2020.
- [6] Carlos A Mendez and Jaime Cerdá. An milp framework for batch reactive scheduling with limited discrete resources. *Computers & Chemical Engineering*, 2004.
- [7] Raine Jokinen, Frank Pettersson, and Henrik Saxén. An milp model for optimization of a small-scale lng supply chain along a coastline. *Applied Energy*, 2015.
- [8] Qingyu Han, Linxin Yang, Qian Chen, Xiang Zhou, Dong Zhang, Akang Wang, Ruoyu Sun, and Xiaodong Luo. A gnn-guided predict-and-search framework for mixed-integer linear programming. In *The International Conference on Learning Representations*, 2023.
- [9] Taoan Huang, Aaron M Ferber, Arman Zharmagambetov, Yuandong Tian, and Bistra Dilkina. Contrastive predict-and-search for mixed integer linear programs. In *International Conference on Machine Learning*, 2024.
- [10] Frank Hutter, Holger H Hoos, Kevin Leyton-Brown, and Thomas Stützle. Paramils: An automatic algorithm configuration framework. *Journal of artificial intelligence research*, 2009.
- [11] Gellért Weisz, Andras Gyorgy, and Csaba Szepesvári. Leapsandbounds: A method for approximately optimal algorithm configuration. In *International Conference on Machine Learning*, 2018.
- [12] Lars Kotthoff. Algorithm selection for combinatorial search problems: A survey. In *Data Mining and Constraint Programming: Foundations of a Cross-Disciplinary Approach*. 2016.
- [13] Zeyuan Ma, Hongshu Guo, Yue-Jiao Gong, Jun Zhang, and Kay Chen Tan. Toward automated algorithm design: A survey and practical guide to meta-black-box-optimization. *arXiv preprint arXiv:2411.00625*, 2024.
- [14] Massimo Regona, Tan Yigitcanlar, Bo Xia, and Rita Yi Man Li. Opportunities and adoption challenges of ai in the construction industry: A prisma review. *Journal of open innovation: technology, market, and complexity*, 2022.
- [15] Marcus Tantakoun, Xiaodan Zhu, and Christian Muise. Llms as planning modelers: A survey for leveraging large language models to construct automated planning models. *arXiv preprint arXiv:2503.18971*, 2025.
- [16] Frank Hutter, Lars Kotthoff, and Marius Lindauer. Algorithm configuration: Challenges, methods and perspectives. Tutorial at IJCAI 2020, 2020. Accessed: 2025-05-11.
- [17] Tobias Achterberg. Constraint integer programming. 2007.
- [18] A. Schrijver. *Theory of Linear and Integer Programming*. John Wiley & Sons, 1998.
- [19] Jiayi Zhang, Chang Liu, Xijun Li, Hui-Ling Zhen, Mingxuan Yuan, Yawen Li, and Junchi Yan. A survey for solving mixed integer programming via machine learning. *Neurocomputing*, 2023.

- 414 [20] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark,
415 AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv*
416 *preprint arXiv:2410.21276*, 2024.
- 417 [21] Tianxing Yang, Huigen Ye, and Hua Xu. Code retrieval for milp instance generation. *arXiv*
418 *preprint arXiv:2505.11526*, 2025.
- 419 [22] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact
420 combinatorial optimization with graph convolutional neural networks. *Advances in neural*
421 *information processing systems*, 2019.
- 422 [23] Jian-Ya Ding, Chao Zhang, Lei Shen, Shengyin Li, Bing Wang, Yinghui Xu, and Le Song.
423 Accelerating primal solution findings for mixed integer programs based on solution prediction.
424 In *AAAI Conference on Artificial Intelligence*, 2020.
- 425 [24] Ziang Chen, Jialin Liu, Xinshang Wang, and Wotao Yin. On representing mixed-integer linear
426 programs by graph neural networks. In *International Conference on Learning Representations*,
427 2023.
- 428 [25] Nicolas Sonnerat, Pengming Wang, Ira Ktena, Sergey Bartunov, and Vinod Nair. Learn-
429 ing a large neighborhood search algorithm for mixed integer programs. *arXiv preprint*
430 *arXiv:2107.10201*, 2021.
- 431 [26] Huigen Ye, Hua Xu, and Hongyan Wang. Light-milpopt: Solving large-scale mixed integer
432 linear programs with lightweight optimizer and small-scale training dataset. In *International*
433 *Conference on Learning Representations*, 2024.
- 434 [27] Tobias Achterberg. What’s new in gurobi 9.0. *Webinar Talk url: [https://www.gurobi.com/wp-](https://www.gurobi.com/wp-content/uploads/2019/12/Gurobi-90-Overview-Webinar-Slides-1.pdf)*
435 *content/uploads/2019/12/Gurobi-90-Overview-Webinar-Slides-1.pdf*, 2019.
- 436 [28] Tobias Achterberg. Scip: Solving constraint integer programs. *Mathematical Programming*
437 *Computation*, 2009.
- 438 [29] Taehyun Yoon. Confidence threshold neural diving. *arXiv preprint arXiv:2202.07506*, 2022.
- 439 [30] Huigen Ye, Hua Xu, Hongyan Wang, Chengming Wang, and Yu Jiang. Gnn & gbdt-guided fast
440 optimizing framework for large-scale integer programming. In *International Conference on*
441 *Machine Learning*, 2023.
- 442 [31] Jialin Song, Yisong Yue, Bistra Dilikina, et al. A general large neighborhood search framework
443 for solving integer linear programs. *Advances in Neural Information Processing Systems*, 2020.
- 444 [32] Gregor Hendel. Adaptive large neighborhood search for mixed integer programming. *Mathe-*
445 *matical Programming Computation*, 2022.
- 446 [33] Vinod Nair, Mohammad Alizadeh, et al. Neural large neighborhood search. In *Learning Meets*
447 *Combinatorial Algorithms at NeurIPS2020*, 2020.
- 448 [34] Timo Berthold. *Primal heuristics for mixed integer programs*. PhD thesis, Zuse Institute Berlin
449 (ZIB), 2006.
- 450 [35] Huigen Ye, Hongyan Wang, Hua Xu, Chengming Wang, and Yu Jiang. Adaptive constraint
451 partition based optimization framework for large-scale integer linear programming. In *AAAI*
452 *Conference on Artificial Intelligence*, 2023.
- 453 [36] Egon Balas and Manfred W Padberg. On the set-covering problem. *Operations Research*, 1972.
- 454 [37] Samir Khuller. Algorithms column: The tertext cover problem. *ACM SIGACT News*, 2002.
- 455 [38] Jan-Henrik Haunert and Alexander Wolff. Beyond maximum independent set: An extended
456 integer programming formulation for point labeling. *ISPRS International Journal of Geo-*
457 *Information*, 2017.
- 458 [39] Alper Atamtürk. Cover and pack inequalities for (mixed) integer programming. *Annals of*
459 *Operations Research*, 2005.

- 460 [40] Kalyanmoy Deb, Samir Agrawal, Amrit Pratap, and Tanaka Meyarivan. A fast elitist non-
461 dominated sorting genetic algorithm for multi-objective optimization: Nsga-ii. In *International*
462 *conference on parallel problem solving from nature*, pages 849–858. Springer, 2000.
- 463 [41] Karl Bringmann and Tobias Friedrich. An efficient algorithm for computing hypervolume
464 contributions. *Evolutionary Computation*, 18(3):383–402, 2010.
- 465 [42] Czesław Smutnicki, Jarosław Rudy, and Dominik Żelazny. Very fast non-dominated sorting.
466 *Decision Making in Manufacturing and Services*, 8(1-2):13–23, 2014.
- 467 [43] Benjamin Blonder, Christine Lamanna, Cyrille Violle, and Brian J Enquist. The n-dimensional
468 hypervolume. *Global Ecology and Biogeography*, 23(5):595–609, 2014.
- 469 [44] Lyndon While, Lucas Bradstreet, and Luigi Barone. A fast way of calculating exact hypervol-
470 umes. *IEEE Transactions on Evolutionary Computation*, 16(1):86–95, 2011.
- 471 [45] Andrzej Jaskiewicz and Piotr Zielniewicz. Quick extreme hypervolume contribution algorithm.
472 In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 412–420,
473 2021.
- 474 [46] Peter I Frazier. A tutorial on bayesian optimization. *arXiv preprint arXiv:1807.02811*, 2018.
- 475 [47] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna:
476 A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM*
477 *SIGKDD international conference on knowledge discovery & data mining*, pages 2623–2631,
478 2019.
- 479 [48] László Erdős, Antti Knowles, Horng-Tzer Yau, and Jun Yin. Spectral statistics of erdős–rényi
480 graphs i: Local semicircle law. 2013.
- 481 [49] Siddharth Pal and Ananthram Swami. Geometric nodal degree distributions arise in barabási-
482 albert graphs! *IEEE Transactions on Network Science and Engineering*, 9(3):1409–1421,
483 2022.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and the final paragraph of the introduction clearly state the paper's contributions, including the formulation of LEGO as a general framework for adaptive algorithm construction and its instantiation in MILP solving. These claims are consistent with the theoretical formulation and empirical results presented in the paper.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: The limitations of our approach are discussed at the end of the Conclusion section, where we acknowledge the framework's reliance on the quality of the problem generator and outline future work to address this limitation by developing more generalizable generation and retrieval mechanisms.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: The paper does not include formal theoretical results or proofs; it focuses on a modular framework and its empirical evaluation.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We provide detailed descriptions of experimental configurations, solver settings, and hyperparameter choices in both the main text (Section 4) and Appendix A. Additionally, we have released a demo of LEGO at <https://anonymous.4open.science/r/LEGO-B36D>, which includes code and instructions to facilitate quick reproduction of our key results.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide open access to a demo implementation of LEGO at <https://anonymous.4open.science/r/LEGO-B36D>, which includes code, example data, and detailed instructions to reproduce the main experimental results. Further reproduction details are also described in the supplemental material, following NeurIPS code and data submission guidelines.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [Yes]

Justification: We provide comprehensive descriptions of the experimental setup in both the main paper (Section 4) and the appendix, including details on datasets, data splits, hyperparameter settings, and selection strategies. In addition, the open-source demo at <https://anonymous.4open.science/r/LEGO-B36D> contains the full codebase and configuration files needed to reproduce the experiments.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We provide error bar analyses based on multiple experimental runs in the appendix, including explanations of the sources of variability (e.g., random seeds) and the method used for computing standard deviations. These results support the robustness and statistical significance of our main empirical findings.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We specify the type of compute workers used (e.g., CPU/GPU configurations) in the experimental setup section of the main paper. An estimate of the total compute cost, including time and resource usage across different experiments, is provided in the appendix.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We have carefully reviewed the NeurIPS Code of Ethics and confirm that our research complies fully with its principles. Our work does not involve human subjects, private or sensitive data, or applications with foreseeable harmful impact. We have followed best practices in dataset usage, code release, and reproducibility, and have taken care to consider potential societal and environmental impacts.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [NA]

Justification: Our work focuses on the development of a general-purpose optimization framework and does not involve data generation, interaction with end-users, or deployment in application-specific contexts. As such, we do not foresee any direct societal impact—positive or negative—at this stage, nor any obvious potential for malicious use.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: Our work does not involve the release of pretrained generative models, large language models, or datasets that carry a high risk of misuse. Therefore, no specific safeguards are necessary.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: All external assets used in our work, including datasets, codebases, and models, are properly cited in the main paper. We have ensured that their licenses and terms of use are respected, and we include references to the original papers and repositories where applicable.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: Our paper introduces new algorithmic components, which we plan to open-source upon publication. A demo showcasing core functionalities is already publicly available and can be freely used. The final release will include detailed documentation covering usage instructions, license, limitations, and implementation details.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[NA\]](#)

Justification: Our work does not involve any form of crowdsourcing or research with human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA] .

Justification: Our research does not involve human subjects or any form of human-subject experimentation, and therefore no IRB or equivalent ethical review was necessary.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Our paper includes an LLM-enhanced performance evaluation component, where large language models are used to generate novel evaluation metrics. The specific models used, their configurations, and their role in the evaluation process are clearly described in the paper, along with appropriate citations.

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (<https://neurips.cc/Conferences/2025/LLM>) for what should or should not be described.

A Overview of Appendix

This appendix provides additional technical and experimental details to support the proposed **LEGO** framework. It is organized into four sections, each focusing on a specific aspect of our method and evaluation.

- **Supplementary Methodology** (Appendix B): This section elaborates on the core techniques introduced in the paper, including the design of handcrafted and LLM-generated evaluation metrics, hypervolume-guided component selection, size-transferable parameter tuning, and detailed descriptions of the component library used to build solving pipelines.
- **Experimental Setup** (Appendix C): This section outlines the mathematical formulations, parameter configurations, and instance scales of the four MILP benchmark problems. It also reports runtime and objective statistics, hyperparameter design strategies, and estimates of computational cost.
- **Additional Results and Analysis** (Appendix D): This section presents extended experimental findings, including MILP-Retrieval results, component ranking and selection outcomes, error bar analysis across scales, and multiple ablation studies to assess the contributions of evaluation metrics, hypervolume ranking, and transferability mechanisms.
- **Future Work Directions** (Appendix E): This section discusses limitations and potential future enhancements, such as reducing dependence on LLM quality, improving scalability and generalization across problem sizes, and exploring data-driven methods for automatic metric selection.

The LEGO framework aims to automatically construct effective and scalable optimization algorithms by adaptively selecting and configuring modular components. It leverages large language models for hybrid evaluation, combines traditional performance measures with LLM-informed assessments, and employs hypervolume-based ranking to identify high-quality solving pipelines. By enabling cross-scale adaptability through synthetic data generation and performance transfer, LEGO demonstrates strong generalization across diverse MILP tasks. This appendix provides the necessary methodological and empirical details to support and reproduce the results presented in the main paper.

B Supplementary Methodology

This section provides supplementary details on the core components of the LEGO framework. It includes four key parts: the LLM-enhanced performance evaluation mechanism, the hypervolume-guided component selection strategy, the size-transferable parameter tuning process, and the design of the modular component library.

We present the design and implementation of handcrafted and LLM-generated evaluation metrics, multi-objective selection based on Pareto hypervolume, parameter tuning strategies using MILP-Retrieval and Bayesian optimization, and detailed descriptions of the four core components used to construct solving pipelines. For each part, we provide pseudocode, complexity analysis, and implementation insights to support reproducibility.

B.1 LLM-enhanced Performance Evaluation

To support fine-grained evaluation and selection of solver pipelines, we develop a hybrid performance evaluation framework that combines human-designed metrics and LLM-generated criteria. This approach captures both classical notions of solution quality and more dynamic, process-level characteristics learned from solver behavior. Specifically, we define a 7-dimensional evaluation vector composed of four handcrafted metrics and three additional metrics derived from large language model (LLM) suggestions.

In this section, we first describe the handcrafted metrics grounded in oracle-based comparisons. We then present the construction pipeline for LLM-generated metrics, including prompt design, output filtering, and human validation. Finally, we introduce the selected LLM-based metrics and provide their formal definitions and implementation details.

Algorithm 1 Initial Solution Gap

```
function INITIALGAP(val_list, x_star)
   $v_0 \leftarrow \text{val\_list}[0]$ 
  if  $v_0 = 0$  then
    return large_penalty
  end if
   $gap \leftarrow |v_0 - x\_star| / x\_star$ 
  return gap
end function
```

Algorithm 2 Final Solution Gap

```
function FINALGAP(val_list, x_star)
   $v_T \leftarrow \text{val\_list}[-1]$ 
  if  $v_T = 0$  then
    return large_penalty
  end if
   $gap \leftarrow |v_T - x\_star| / x\_star$ 
  return gap
end function
```

893 B.1.1 Handcrafted Evaluation Metrics

894 To provide a reliable and interpretable evaluation of solver performance, we define four classical
895 metrics based on oracle-level comparisons. These handcrafted metrics are designed to capture both
896 the effectiveness and efficiency of the solving process. For each instance, we run a strong solver to
897 obtain a near-optimal lower bound x^* , and evaluate candidate pipelines using their time-series outputs:
898 a list of timestamps and corresponding objective values, denoted as (time_list, val_list).

899 **(1) Initial Solution Gap.** This metric evaluates the quality of the first solution reported by the
900 solver, typically produced by the *predict* stage in LEGO’s Predict-and-Search pipeline. Since the
901 initial solution is generated before any search-based refinement, it directly reflects the accuracy and
902 effectiveness of the predictive model or heuristic used to initialize the solving process.

903 As shown in Algorithm 1, we compute the relative gap between the predicted initial solution v_0 and
904 the oracle lower bound x^* , defined as: $\text{InitialGap} = \frac{|v_0 - x^*|}{x^*}$.

905 A smaller initial gap indicates that the prediction is close to optimal, which serves as a strong
906 foundation for subsequent improvement steps in the search phase. Therefore, this metric provides a
907 focused evaluation of the *predict* module’s standalone performance, independent of any downstream
908 optimization.

909 **Evaluation Type:** Effectiveness **Input:** val_list[0], x^* **Computation:** $gap = \frac{|v_0 - x^*|}{x^*}$

910 **(2) Final Solution Gap.** This metric evaluates the quality of the final solution obtained by a solver
911 pipeline at the end of the allowed time budget. Unlike the initial gap, which targets the performance
912 of the prediction phase, the final gap reflects the *integrated effectiveness* of the entire solving process,
913 including both the initial prediction and all subsequent search-based refinement steps.

914 As shown in Algorithm 2, let v_T denote the last solution in the objective trace, and x^* the oracle
915 lower bound. The final gap is computed as: $\text{FinalGap} = \frac{|v_T - x^*|}{x^*}$.

916 A smaller final gap indicates that the solver has successfully improved upon the initial prediction and
917 approached the optimal solution. In the LEGO framework, this metric serves as a comprehensive
918 indicator of end-to-end pipeline quality, capturing the joint effect of component selection, parameter
919 tuning, and the interaction between prediction and search stages.

920 **Evaluation Type:** Effectiveness **Input:** val_list[-1], x^* **Computation:** $gap = \frac{|v_T - x^*|}{x^*}$

Algorithm 3 Efficiency Rate

```
function EFFICIENCYRATE(val_list, x_star)
    count ← 0
    for all v in val_list do
        if |v - x_star| / x_star < 0.1 then
            count ← count + 1
        end if
    end for
    return count / len(val_list)
end function
```

Algorithm 4 Time to Valid and High-Quality Solutions

```
function TIMETOSOLUTIONS(time_list, val_list, x_star)
    t10 ← 999999
    t1 ← 999999
    for i = 0 to len(val_list) - 1 do
        gap ← |val_list[i] - x_star| / x_star
        if gap < 0.1 and t10 = 999999 then
            t10 ← time_list[i]
        end if
        if gap < 0.01 and t1 = 999999 then
            t1 ← time_list[i]
        end if
    end for
    return t10 + t1
end function
```

921 **(3) Efficiency Rate.** This metric assesses the solver’s ability to *consistently* produce acceptable-
922 quality solutions throughout the solving horizon. Rather than focusing on only the first or final
923 solution, it captures how reliably the solver maintains a desirable performance level over time.
924 Specifically, we define an “acceptable” solution as one with a gap to the oracle lower bound x^*
925 smaller than 10%, i.e., $|v_t - x^*| / x^* < 0.1$.

926 Given a sequence of solver checkpoints $(v_1, v_2, \dots, v_T)$, the efficiency rate is computed as
927 the proportion of these checkpoints satisfying the acceptable gap condition: $\text{EfficiencyRate} =$
928 $\frac{1}{T} \sum_{t=1}^T \mathbf{1} \left[\frac{|v_t - x^*|}{x^*} < 0.1 \right]$.

929 As shown in Algorithm 3, this metric reflects the solver’s practical usefulness in time-sensitive settings,
930 where intermediate solutions may be deployed before the time limit. In the LEGO framework, it
931 provides insight into the effectiveness of the *search-phase components*, particularly those responsible
932 for trajectory shaping, local improvement, and feasibility repair. A higher efficiency rate indicates
933 more stable behavior and better anytime performance.

934 **Evaluation Type:** Efficiency **Input:** (val_list, x^*) **Computation:** rate =
935 $\frac{1}{T} \sum_{t=1}^T \mathbf{1} \left[\frac{|v_t - x^*|}{x^*} < 0.1 \right]$

936 **Evaluation Type:** Efficiency **Input:** (time_list, val_list, x^*) **Computation:** Let $t_{10\%}$ and $t_{1\%}$
937 be the first times such that the gap drops below 10% and 1%, respectively.

938 **(4) Time to Valid and High-Quality Solutions.** This metric measures the solver’s responsiveness
939 by evaluating how quickly it is able to discover solutions of acceptable and exceptional quality.
940 Specifically, we track two key milestones in the solution trajectory: (i) the first *valid solution*, defined
941 as achieving a relative gap to the oracle lower bound x^* below 10%, and (ii) the first *high-quality*
942 *solution*, where the gap drops below 1%.

943 Let $t_{10\%}$ and $t_{1\%}$ denote the timestamps at which these two thresholds are first reached, respectively.
944 The combined score is computed as: $\text{TimeScore} = t_{10\%} + t_{1\%}$.

As shown in Algorithm 4, a lower score indicates that the solver rapidly identifies both acceptable and high-quality solutions, which is particularly important in real-time or resource-constrained settings. In the context of LEGO, this metric is useful for evaluating the *search components*—such as feasibility repair or local search—that directly influence the early-stage and mid-stage progression of the solution trajectory. Moreover, it reflects the solver’s *anytime behavior*, i.e., how useful its intermediate outputs are before convergence.

B.1.2 LLM Prompt Design and Metric Selection Pipeline

To complement handcrafted metrics with more diverse and statistically meaningful evaluation criteria, we leverage large language models (LLMs) to automatically generate candidate metrics based on solver trajectory data. Our goal is to explore novel ways to assess solver behavior, particularly those that capture dynamic patterns beyond initial and final performance.

We interact with GPT-4o [20] via API and design structured prompts that provide the LLM with existing handcrafted metric implementations (e.g., initial gap, final gap, efficiency rate), along with detailed descriptions of their statistical intentions. We then explicitly ask the LLM to propose new evaluation metrics that are *distinct from the provided ones*, and that reflect *process-level statistical characteristics* of the solution trajectory. Each prompt includes Python-style code snippets, performance traces, and clarifications on the evaluation setting.

The LLM responds with a variety of metric candidates, expressed in natural language explanations and executable pseudocode. From multiple prompt rounds, we collect and curate a set of 15 candidate metrics. These include:

The following is a representative list of LLM-suggested metric candidates:

- **Convergence Stability:** Measures the sign-change frequency in the objective value trend. A low value indicates smoother convergence with fewer oscillations.
- **Early Progress Rate:** Quantifies how much improvement is made in the early 10% of the solving process. Faster early progress yields a better score.
- **Mid-stage Stagnation:** Detects flat regions in the middle 40–60% segment of the solution trajectory. High stagnation implies unproductive mid-phase behavior.
- **Late-stage Convergence Index:** Captures how much improvement occurs in the final 20% of the time budget. High values indicate poor late-stage convergence.
- **Overall Improvement Efficiency:** Computes the ratio of time used to objective improvement, mapped via a logarithmic normalization. Lower values are better.
- **Other Statistical Metrics:**
 - **Relative Gap Improvement** — normalized difference between initial and final gap.
 - **Area Under the Curve (AUC)** — time-integrated solution gap over the trajectory.
 - **Time to 90% Progress** — time taken to achieve 90% of total improvement.
 - **Early Convergence Ratio** — how close the solution gets to final quality within the first 10% of time.
 - **Consistency** — rate of monotonic improvement violations.
 - **Monotonicity** — whether solution and time sequences are monotonic.
 - **Time Efficiency** — total time normalized via log scaling.

Each metric is further analyzed in terms of its computational form (e.g., time-normalized, log-scaled), interpretability, and alignment with solver behaviors.

After filtering metrics with high redundancy, unclear meaning, or low robustness across validation trajectories, we manually select three metrics that are most informative about the *process dynamics* rather than just endpoints. These metrics are:

- **Solution gap at 20% of the time budget,**
- **Solution gap at 60% of the time budget,**
- **Time-integrated solution gap over the full horizon (area under the curve).**

Algorithm 5 Gap at 20% Time

```
function GAPAT20%TIME(time_list, val_list, x_star)
   $th \leftarrow 0.2 \cdot \text{time\_list}[-1]$ 
   $idx \leftarrow -1$ 
  for  $i = 0$  to  $\text{len}(\text{time\_list}) - 1$  do
    if  $\text{time\_list}[i] < th$  then
       $idx \leftarrow i$ 
    end if
  end for
  if  $idx = -1$  then
    return large_penalty
  end if
   $v \leftarrow \text{val\_list}[idx]$ 
   $gap \leftarrow |v - x_{star}| / x_{star}$ 
  return gap
end function
```

993 These metrics are chosen to enrich the evaluation space with trajectory-aware, intermediate-stage
994 signals that complement traditional start/end-point metrics.

995 In the next section, we provide formal definitions, implementation details, and pseudocode for each
996 of the selected LLM-derived indicators.

997 B.1.3 Selected LLM-derived Metrics

998 To complement the handcrafted metrics, we incorporate process-oriented indicators proposed by a
999 large language model (GPT-4o). After filtering and human validation, we retain three trajectory-
1000 sensitive metrics that capture intermediate quality and convergence characteristics: (5) solution gap at
1001 20% of the time budget, (6) solution gap at 60%, and (7) the time-integrated gap over the full horizon.

1002 **(5) Gap at 20% Time.** This metric evaluates the solver’s early progress by measuring the relative
1003 gap between the intermediate solution (at 20% of the total time budget) and the oracle lower bound x^* .
1004 It reflects how quickly the solver escapes poor initial solutions and begins meaningful convergence.

1005 As shown in Algorithm 5, we locate the latest evaluation point whose timestamp is lower than $0.2 \cdot T$,
1006 and compute its gap. This approach avoids interpolation and ensures robustness on sparse logs.

$$\text{Gap}_{20\%} = \frac{|v_{20\%} - x^*|}{x^*}$$

1007 **Evaluation Type:** Intermediate Effectiveness **Input:** (time_list, val_list, x^*) **Computation:**
1008 Relative gap at 20% time

1009 **(6) Gap at 60% Time.** This metric examines the solution quality in the mid-phase of the search
1010 process. It is particularly useful for detecting mid-stage stagnation or plateauing behavior.

1011 As shown in Algorithm 6, similar to the previous metric, we compute the gap at the latest time step
1012 before 60% of the time budget, ensuring the method is applicable even when evaluations are sparsely
1013 logged.

$$\text{Gap}_{60\%} = \frac{|v_{60\%} - x^*|}{x^*}$$

1014 **Evaluation Type:** Mid-stage Effectiveness **Input:** (time_list, val_list, x^*) **Computation:**
1015 Relative gap at 60% time

1016 **(7) Area Under the Gap-Time Curve.** This metric quantifies the overall convergence process by
1017 integrating the normalized gap over time. It captures both speed and quality of convergence—lower
1018 values indicate faster and smoother progress toward the optimum.

Algorithm 6 Gap at 60% Time

```
function GAPAT60%TIME(time_list, val_list, x_star)
  th ← 0.6 · time_list[-1]
  idx ← -1
  for i = 0 to len(time_list)-1 do
    if time_list[i] < th then
      idx ← i
    end if
  end for
  if idx = -1 then
    return large_penalty
  end if
  v ← val_list[idx]
  gap ← |v - x_star| / x_star
  return gap
end function
```

Algorithm 7 Area Under the Gap-Time Curve

```
function GAPAUC(time_list, val_list, x_star)
  auc ← 0
  for i = 1 to len(time_list)-1 do
    dt ← time_list[i] - time_list[i-1]
    g1 ← |val_list[i] - x_star| / x_star
    g0 ← |val_list[i-1] - x_star| / x_star
    auc ← auc + dt · (g0 + g1) / 2
  end for
  return auc
end function
```

1019 As shown in Algorithm 7, since we only observe discrete solver checkpoints, we approximate the
1020 integral using the trapezoidal rule applied to adjacent gap-time pairs. This linear interpolation is
1021 numerically stable and efficient.

$$\text{AUC} = \int_0^T g(t) dt \quad (\text{approximated via trapezoidal rule})$$

1022 **Evaluation Type:** Global Efficiency **Input:** (time_list, val_list, x^*) **Computation:** Gap
1023 integral over time

1024 B.2 Hypervolume-guided Component Selection

1025 To efficiently select promising solver pipelines from the large combinatorial space of LEGO com-
1026 ponent configurations, we evaluate all candidates on small-scale training instances using the *LLM-*
1027 *enhanced Performance Evaluation* described earlier. This yields a 7-dimensional performance vector
1028 for each candidate, incorporating both handcrafted and LLM-derived metrics.

1029 B.2.1 Pareto Frontier Identification

1030 We begin by applying non-dominated sorting to identify the Pareto frontier S , which contains only
1031 those pipelines that are not strictly outperformed in all metrics [40]. Formally, a configuration a is
1032 said to dominate b if a is no worse than b on all dimensions and strictly better on at least one. The
1033 frontier S thus represents the set of candidates offering optimal trade-offs among multiple objectives.

Algorithm 8 Pareto Frontier Extraction

```
function GETPARETOFRONT( $D$ )
   $S \leftarrow \emptyset$ 
  for all  $d \in D$  do
     $is\_dominated \leftarrow \text{False}$ 
    for all  $d' \in D$  do
      if  $d' \neq d$  and  $\text{DOMINATES}(d', d)$  then
         $is\_dominated \leftarrow \text{True}$ 
        break
      end if
    end for
    if not  $is\_dominated$  then
       $S \leftarrow S \cup \{d\}$ 
    end if
  end for
  return  $S$ 
end function

function DOMINATES( $a, b$ )
   $better \leftarrow \text{False}$ 
  for  $i = 1$  to  $n$  do
    if  $a[i] > b[i]$  then return False
    end if
    if  $a[i] < b[i]$  then  $better \leftarrow \text{True}$ 
    end if
  end for
  return  $better$ 
end function
```

Algorithm 9 Hypervolume Contribution Ranking

```
function RANKBYHYPERVOLUME( $S, ref\_point$ )
   $total\_hv \leftarrow \text{COMPUTEHV}(S, ref\_point)$ 
   $scores \leftarrow \text{empty list}$ 
  for all  $d \in S$  do
     $S' \leftarrow S \setminus \{d\}$ 
     $hv\_drop \leftarrow total\_hv - \text{COMPUTEHV}(S', ref\_point)$ 
    append ( $d, hv\_drop$ ) to  $scores$ 
  end for
  return  $scores$  sorted by  $hv\_drop$  descending
end function
```

1034 B.2.2 Hypervolume Contribution Ranking

1035 While Pareto filtering removes clearly suboptimal configurations, the remaining set S may still
1036 contain many similar or marginally different candidates [41]. To further prioritize configurations, we
1037 compute the *hypervolume contribution* $\Delta V(d)$ of each configuration $d \in S$:

$$\Delta V(d) = V(S) - V(S \setminus \{d\}),$$

1038 where $V(\cdot)$ denotes the hypervolume under a fixed reference point. Intuitively, $\Delta V(d)$ measures how
1039 much unique volume in objective space is covered by d alone. Configurations with large contributions
1040 are more diverse and valuable for maintaining performance complementarity.

1041 B.2.3 Complexity Analysis

1042 We analyze the computational complexity of the full component selection pipeline. Let N be the total
1043 number of candidate pipelines (typically $N \approx 200$), and n the number of evaluation metrics (here,
1044 $n = 7$). The process consists of two main steps:

1045 • **Pareto Frontier Computation:** As shown in Algorithm 8, each configuration is compared
 1046 against all others to check for dominance. The worst-case complexity [42] is:

$$O(N^2 \cdot n),$$

1047 since we perform $O(N^2)$ comparisons and each comparison costs $O(n)$. In practice, this
 1048 step is fast due to small n and vectorized comparison implementations.

1049 • **Hypervolume Contribution Computation:** For each configuration $d \in S$ on the Pareto
 1050 frontier, we compute the hypervolume of S and of $S \setminus \{d\}$ to obtain $\Delta V(d)$. The complexity
 1051 depends on the size of the frontier $|S|$ and the number of dimensions n :

- 1052 – Computing the hypervolume exactly in n dimensions has worst-case complexity of
 1053 $O(|S|^{n-1})$ [43].
- 1054 – More efficient algorithms (e.g., WFG [44], QHV [45]) reduce this to $O(|S| \cdot \log |S|)$
 1055 for small n , but still scale poorly with dimensionality.

1056 Fortunately, in our experiments:

- 1057 – The total number of candidates is moderate ($N \approx 200$),
- 1058 – The number of non-dominated solutions is small ($|S| \leq 20$ in most cases),
- 1059 – The number of metrics is fixed and low ($n = 7$).

1060 As a result, the overall cost remains manageable, especially since we only run this selection
 1061 on small-scale training instances.

1062 B.2.4 Final Selection Strategy

1063 We rank all candidates in S by their hypervolume contribution and select the top- K configurations
 1064 for further tuning and deployment. This strategy ensures that the final set of solver pipelines is both
 1065 *high-performing* and *diverse*, and avoids redundant configurations that offer little marginal value.

1066 B.3 Size-transferable Parameter Tuning

1067 While component selection is performed on small-scale training instances, we introduce a dedicated
 1068 *Size-transferable Parameter Tuning* stage to ensure that selected solver pipelines generalize to larger-
 1069 scale MILP problems. This stage jointly finalizes both the component choices and their associated
 1070 hyperparameters, preparing the solvers for deployment.

1071 B.3.1 MILP-Retrieval for Controllable Instance Generation

1072 In many practical scenarios, the available training data may only contain small-scale MILP instances.
 1073 To address this limitation, we leverage the MILP-Retrieval [21] framework, which enables controllable
 1074 generation of structurally similar MILP instances across different scales and difficulties.

1075 MILP-Retrieval retrieves formulation code from a diverse MILP library by matching embedding
 1076 similarity between the target and library instances. This similarity is computed using a pretrained
 1077 MILP embedding model based on contrastive learning. The retrieved formulation code is then *tuned*
 1078 via parameter perturbation (e.g., number of variables, constraint density) to synthesize new instances
 1079 that preserve semantic structure while varying in size.

1080 This mechanism allows us to simulate large-scale conditions even when only small-scale instances
 1081 are available, enabling effective parameter tuning under limited data regimes.

1082 B.3.2 Bayesian Optimization for Hyperparameter Selection

1083 For each of the top- K component configurations selected via hypervolume-guided comparison, we
 1084 perform hyperparameter tuning using **Bayesian Optimization** [46]. Formally, let $f(\theta)$ denote the
 1085 performance score of a solver configuration with hyperparameters θ , as evaluated by the 7-dimensional
 1086 performance vector introduced in Appendix B.1. The goal is to find:

$$\theta^* = \arg \min_{\theta \in \Theta} \mathbb{E}[f(\theta)],$$

1087 where Θ denotes the feasible hyperparameter space.

1088 We implement this optimization using the Optuna library [47], which provides an efficient and
 1089 extensible framework for black-box optimization. Each evaluation of $f(\theta)$ involves solving a batch
 1090 of MILP instances and computing their LLM-enhanced performance metrics.

1091 To ensure robustness and scalability, the tuning process is performed on a mixture of:

- 1092 • **Real training instances** (if available at large scale),
- 1093 • **Synthetic large-scale instances** generated by MILP-Retrieval.

1094 B.3.3 Final Selection and Deployment

1095 After tuning, each candidate pipeline is re-evaluated using the full metric suite. We then apply the
 1096 same hypervolume-based ranking method described in Section B.2 to identify the best-performing
 1097 configuration.

1098 The final output of LEGO is thus a solver pipeline whose components and hyperparameters are jointly
 1099 optimized for efficiency, effectiveness, and scale-transferability.

1100 B.4 Component Library of Predict-and-Search

1101 Building on the LLM-based evaluation and self-optimization framework introduced above, we apply
 1102 LEGO to the important application domain of Mixed-Integer Linear Programming (MILP) [17].
 1103 Specifically, we instantiate LEGO within the widely adopted *Predict-and-Search* (P&S) paradigm
 1104 [8], a dominant architecture in learning-based MILP solving.

1105 Under this paradigm, the solver pipeline is decomposed into four functional stages:

- 1106 1. **Graph Embed** — transforms MILP instances into structured representations suitable for
 1107 learning;
- 1108 2. **Neural Predict** — produces an initial feasible or near-feasible solution using learned
 1109 models;
- 1110 3. **Feasibility Repair** — ensures hard constraint satisfaction via projection or local repair;
- 1111 4. **Iterative Search** — improves the solution via search-based refinement techniques.

1112 Each stage defines a modular interface with multiple interchangeable implementations. This design
 1113 forms the LEGO *Component Library*, allowing the system to dynamically compose and evaluate
 1114 diverse solver pipelines. The modularity also ensures that new methods can be easily added or
 1115 replaced without modifying the overall architecture.

1116 In the following subsections, we describe the design and implementation of each component group,
 1117 including algorithmic logic, complexity analysis, and pseudocode.

1118 B.4.1 Graph Embed

1119 Directly feeding the raw algebraic form of a MILP problem into a neural model may obscure key
 1120 structural invariances, such as row and column permutations that preserve problem equivalence. To
 1121 retain these desirable symmetries, graph-based representations are commonly adopted, as they are
 1122 inherently invariant to permutations of node order and layout.

1123 The role of the *Graph Embed* module is to transform a given MILP instance into a lossless graph
 1124 representation that preserves its combinatorial and constraint structure, while exposing features
 1125 suitable for downstream neural processing.

1126 In LEGO, we support four commonly used graph encodings, each forming a distinct component in
 1127 the Graph Embed stage of our Component Library:

- 1128 • **Bipartite Graph** [22]: Represents variables and constraints as two disjoint sets of nodes.
 1129 An edge exists between variable x_i and constraint c_j if x_i participates in c_j . This encoding
 1130 is widely used due to its simplicity and lossless structure preservation.
- 1131 • **Tripartite Graph** [23]: Introduces a third node type for objective coefficients, enabling
 1132 explicit modeling of optimization direction (e.g., minimization vs. maximization). This
 1133 representation is beneficial for tasks where objective-related signals are important.

Algorithm 10 Graph Construction Interface

```
function GRAPHEMBED( $A, b, c, l, u, I, \text{mode}$ )  
  if  $\text{mode} = \text{"Bipartite Graph"}$  then  
     $V \leftarrow \text{variables} \cup \text{constraints}$   
     $E \leftarrow \{(x_i, c_j) \mid A[j, i] \neq 0\}$   
     $X \leftarrow \text{node features from variable and constraint metadata}$   
  else if  $\text{mode} = \text{"Tripartite Graph"}$  then  
     $V \leftarrow \text{variables} \cup \text{constraints} \cup \text{objectives}$   
     $E \leftarrow \{(x_i, c_j), (x_i, \text{obj})\}$   
     $X \leftarrow \text{augmented with objective features}$   
  else if  $\text{mode} = \text{"Bipartite with Random Features"}$  then  
     $G \leftarrow \text{GRAPHEMBED}(A, b, c, l, u, I, \text{"bipartite"})$   
    Add random Gaussian noise to  $X$   
  else if  $\text{mode} = \text{"Tripartite with Random Features"}$  then  
     $G \leftarrow \text{GRAPHEMBED}(A, b, c, l, u, I, \text{"tripartite"})$   
    Add random Gaussian noise to  $X$   
  end if  
  return ( $V, E, X$ )  
end function
```

- 1134 • **Bipartite with Random Features** [24]: Augments the bipartite graph with randomized
1135 node features, enhancing representation diversity. This is particularly useful in *foldable*
1136 MILPs, where structural repetitiveness may hinder learning.
- 1137 • **Tripartite with Random Features** [24]: Combines the tripartite structure with random
1138 feature injection, further increasing expressiveness for semantically repetitive formulations.

1139 Each encoding defines a transformation from MILP algebraic data (A, b, c, l, u, I) into a graph object
1140 $G = (V, E, X)$, where V is the set of nodes, E is the set of edges, and X denotes node/edge features.

1141 **Complexity Analysis** Let n be the number of variables, m the number of constraints, and nnz the
1142 number of nonzero entries in the constraint matrix A :

- 1143 • **Graph construction time:** $O(nnz)$ — we scan all non-zero coefficients to construct edges.
- 1144 • **Node feature construction:** $O(n + m)$ — features are typically extracted from metadata
1145 (e.g., bounds, types).
- 1146 • **Random feature injection:** $O(n + m)$ — each node receives a sampled vector.

1147 **Summary** By modularizing the graph encoding choices, LEGO enables flexible experimentation
1148 with different representational strategies. This is especially valuable when applying neural models
1149 across heterogeneous MILP classes, where structural characteristics can vary widely. All Graph
1150 Embed components conform to a unified interface and can be seamlessly swapped during pipeline
1151 construction and tuning.

1152 B.4.2 Neural Predict

1153 Given the graph representation of a MILP instance, the *Neural Predict* module learns to produce an
1154 initial solution that serves as a warm start or guidance for downstream search. This stage plays a
1155 pivotal role in the Predict-and-Search paradigm by bridging structural representation and solution
1156 space.

1157 **Learning-based Predictors.** LEGO supports multiple learning-based predictors, each implemented
1158 as a neural model that maps graph-structured inputs to variable assignments:

- 1159 • **Graph Convolutional Network (GCN)** [25]: A semi-convolutional GNN that propagates
1160 local features across the bipartite or tripartite MILP graph. Suitable for capturing local
1161 constraint-variable interactions.

Algorithm 11 Neural Predict Interface

```
function NEURALPREDICT(MILP-Graph  $G$ , mode)
  if mode in {"GCN", "GAT"} then
     $model \leftarrow$  Load or train GNN model
     $x \leftarrow$  GNN inference on  $G$ 
    return  $x$  as initial solution candidate
  else if mode in {"Gurobi", "SCIP"} then
     $solver \leftarrow$  Initialize solver with time limit
     $x \leftarrow$  Run solver and extract first feasible solution
    return  $x$ 
  end if
end function
```

- 1162 • **Graph Attention Network (GAT)** [26]: Leverages multi-head attention to weigh relational
1163 dependencies between nodes. Offers better adaptability to heterogeneous graph structures.

1164 All GNN-based predictors are trained using supervised labels derived from near-optimal solutions.
1165 Depth, hidden size, and feature type (e.g., random features) are exposed as tunable hyperparameters.

1166 **Solver-based Predictors.** In difficult settings where neural models may struggle—such as extremely
1167 sparse feasible regions—LEGO supports robust fallback strategies using traditional MIP solvers:

- 1168 • **Gurobi** [27], **SCIP** [28] are supported as black-box predictors. These solvers generate a
1169 feasible solution within a fixed time limit, ensuring initialization success in challenging
1170 regimes.
1171 • These methods are deterministic, tunable via solver parameters, and offer consistent perfor-
1172 mance across problem classes.

1173 **Complexity Analysis.** Let n be the number of variables, m the number of constraints, and nnz the
1174 number of non-zeros in the MILP matrix. We assess complexity in two stages:

- 1175 • **GNN inference:** Each forward pass is $O(L \cdot nnz \cdot d)$ where L is the number of layers, and
1176 d is the hidden dimension. Efficient batching amortizes cost over multiple instances.
1177 • **Solver-based inference:** In theory, exact MILP solvers perform a branch-and-bound (B&B)
1178 search, which has exponential worst-case complexity due to the combinatorial nature of
1179 the solution space. However, in practice, solvers are run under a time budget T , and the
1180 computational cost is typically bounded by $O(T)$ wall-clock time.

1181 **Summary.** By modularizing both learning-based and solver-based predictors, LEGO provides a
1182 flexible and robust initialization mechanism. Neural predictors excel in generalization and speed,
1183 while solver-based methods offer reliability. This dual-track design ensures that the Neural Predict
1184 stage adapts to a wide spectrum of MILP tasks.

1185 B.4.3 Feasibility Repair

1186 The predictions generated by the Neural Predict module are not guaranteed to satisfy all constraints
1187 of the original MILP problem. Directly using such infeasible solutions in downstream search often
1188 leads to inefficient or invalid trajectories. To address this, the *Feasibility Repair* module transforms
1189 potentially infeasible predictions into valid solutions that satisfy all problem constraints.

1190 LEGO supports three complementary repair strategies, each designed to balance prediction guidance
1191 and feasibility enforcement:

- 1192 • **Adaptive Radius Search** [8]: Defines a dynamic “trust region” centered around the pre-
1193 dicted solution. A solver is invoked to explore feasible solutions within a bounded distance
1194 (e.g., number of variable flips) from the prediction.
1195 • **Adaptive Thresholding** [29]: Adjusts the confidence cutoffs for variable assignments.
1196 Variables with low confidence are left unresolved and handled by a lightweight solver.

Algorithm 12 Feasibility Repair Interface

```
function FEASIBILITYREPAIR(Predicted Solution  $x$ , mode)
  if mode = "Adaptive Radius Search" then
    Fix top- $k$  variables by confidence
    Add trust region constraint to MILP
    Solve using solver within radius
  else if mode = "Adaptive Thresholding" then
    Select variables above threshold
    Solve partial MILP with remaining free variables
  else if mode = "Neighborhood Repair" then
    Detect violated constraints
    Prune variables contributing to infeasibility
    Solve repaired MILP
  end if
  return Feasible solution  $x'$ 
end function
```

- 1197 • **Neighborhood Repair** [30]: Identifies locally violated constraints and removes variables
1198 that contribute to infeasibility. This method prunes the solution space and invokes a solver
1199 to repair the remaining variables.

1200 All three strategies are modular and interchangeable, and can be selected automatically by LEGO
1201 depending on the problem domain or empirical performance.

1202 **Complexity Analysis** Let n be the number of variables and m the number of constraints:

- 1203 • **Adaptive Radius**: Adds $O(k)$ auxiliary variables and constraints; invokes full MILP solver
1204 once. Time complexity dominated by solver runtime, bounded by time limit T .
- 1205 • **Adaptive Threshold**: Performs up to $O(\log n)$ iterations with shrinking variable sets. Each
1206 iteration invokes a MILP solver.
- 1207 • **Neighborhood Repair**: Detects violations in $O(mk)$ and repairs with up to 3 solver calls.
1208 Suitable for sparse, locally structured problems.

1209 **Summary** The Feasibility Repair module ensures that downstream search algorithms operate on
1210 valid, constraint-satisfying initializations. By supporting multiple repair strategies, LEGO can adapt
1211 to diverse MILP domains—ranging from dense, strongly coupled problems to sparse, locality-driven
1212 ones. All repair methods conform to a unified interface and can be tuned or selected automatically as
1213 part of the LEGO optimization process.

1214 B.4.4 Iterative Search

1215 Once a feasible solution is obtained, it can be further refined via iterative search, a widely adopted
1216 strategy in modern MILP solving frameworks. The core idea is to fix a subset of decision variables and
1217 iteratively re-optimize over a selected neighborhood using a solver. This local refinement improves
1218 solution quality while preserving feasibility.

1219 LEGO supports five representative strategies at this stage, each implemented as a distinct component
1220 in the Search module:

- 1221 • **Large Neighborhood Search (LNS)** [31]: Randomly selects blocks of variables and re-
1222 optimizes over them using a solver. This provides wide exploration while maintaining
1223 tractability.
- 1224 • **Adaptive LNS (NALNS)** [32]: Dynamically adjusts neighborhood size and block structure
1225 based on recent progress. This improves efficiency and escape from local minima.
- 1226 • **Least Integral Heuristic (LIH)** [33]: Focuses refinement on variables furthest from inte-
1227 grality in the LP relaxation, targeting areas with high uncertainty.

Algorithm 13 Iterative Search Interface

```
function ITERATIVESEARCH(Feasible Solution  $x$ , mode)
  if mode = "LNS" then
    Randomly select neighborhood of variables
    Re-optimize using Gurobi
  else if mode = "NALNS" then
    Adaptively tune block size and region
    Use greedy MILP solver to refine
  else if mode = "LIH" or "MIH" then
    Sort variables by LP integrality
    Re-optimize top-K subset
  else if mode = "ACP" then
    Partition constraints by correlation
    Solve selected subproblems adaptively
  else if mode = "Gurobi" or "SCIP" then
    Feed  $x$  as warm-start to solver
    Let solver refine using native heuristics
  end if
  return Refined solution  $x'$ 
end function
```

- 1228 • **Most Integral Heuristic (MIH)** [34]: Prioritizes variables closest to integrality, aiming for
1229 conservative improvements within a stable subspace.
- 1230 • **Adaptive Constraint Partition (ACP)** [35]: Partitions the MILP by correlated constraints
1231 and performs block-wise optimization, enabling structure-aware search.

1232 **Solver-Guided Search Integration** In addition to local search heuristics, LEGO allows using
1233 the repaired solution as a warm-start for commercial solvers such as **Gurobi** [27] and **SCIP** [28].
1234 These solvers accept the predicted solution as an initial incumbent and refine it using their internal
1235 heuristics and branching strategies. This dual-mode design—search-based refinement and solver-
1236 based integration—makes the Iterative Search stage highly flexible and extensible.

1237 **Complexity Analysis** Assume n variables, m constraints:

- 1238 • **LNS / NALNS / ACP:** Each iteration solves a sub-MILP on $O(n/K)$ variables. With
1239 K blocks and R rounds, complexity is roughly $O(R \cdot T)$, where T is the solver time per
1240 subproblem.
- 1241 • **LIH / MIH:** Sort variables by integrality and re-optimize $O(k)$ variables. Complexity is
1242 $O(n \log n + T)$ with light overhead.
- 1243 • **Gurobi / SCIP warm-start:** One call to solver with initial incumbent, complexity deter-
1244 mined by solver timeout T .

1245 **Summary** The Iterative Search module enables LEGO to continuously refine solutions after initial
1246 prediction and repair. By supporting a diverse set of heuristics and solver integrations, this module
1247 balances generality with specialization, and plays a key role in reaching high-quality MILP solutions
1248 under tight time budgets.

1249 C Experimental Setup

1250 To ensure reproducibility and provide transparency for our evaluation, we present detailed descriptions
1251 of the experimental setup used throughout the paper. This includes the mathematical formulations
1252 and parameter configurations of the benchmark MILP problems, runtime and objective statistics,
1253 hyperparameter search design, and an analysis of computational cost. These details are critical for
1254 understanding the scope and limitations of our results, and for establishing fair comparisons across
1255 different solver pipelines.

1256 C.1 Benchmark Problem Specifications

1257 We consider four representative classes of NP-hard MILP problems: Set Covering (SC), Minimum
 1258 Vertex Cover (MVC), Maximum Independent Set (MIS), and Mixed 0-1 Knapsack Set (MKS). Each
 1259 problem is formulated as a minimization task and evaluated under two representative scales: one with
 1260 roughly 100K variables/constraints (e.g., SC₁), and one with approximately 1M-scale (e.g., SC₂).
 1261 Below, we describe the mathematical formulations and modeling assumptions for each problem class.

1262 **Set Covering (SC)** [36]: This problem models classical coverage tasks such as facility location or
 1263 crew scheduling. Given a universal set $\mathcal{U} = \{1, 2, \dots, n\}$ and a collection of subsets $S_1, \dots, S_m$,
 1264 each with cost c_i , the goal is to cover $\mathcal{U}$ using a minimal-cost selection of subsets.

$$\begin{aligned}
 \min \quad & \sum_{i=1}^m c_i x_i \\
 \text{s.t.} \quad & \sum_{i: j \in S_i} x_i \geq 1, \quad \forall j \in \mathcal{U}, \\
 & x_i \in \{0, 1\}, \quad \forall i.
 \end{aligned} \tag{2}$$

1265 **Minimum Vertex Cover (MVC)** [37]: MVC arises in applications such as network monitoring and
 1266 resource allocation. Given an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, the goal is to select the smallest subset of
 1267 vertices such that every edge is incident to at least one selected node.

$$\begin{aligned}
 \min \quad & \sum_{v \in \mathcal{V}} x_v \\
 \text{s.t.} \quad & x_u + x_v \geq 1, \quad \forall (u, v) \in \mathcal{E}, \\
 & x_v \in \{0, 1\}, \quad \forall v \in \mathcal{V}.
 \end{aligned} \tag{3}$$

1268 **Maximum Independent Set (MIS)** [38]: The MIS problem aims to find the largest set of mutually
 1269 non-adjacent nodes in a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, and is widely studied in wireless communication and social
 1270 network analysis. Formally, the original objective is to maximize the cardinality of the independent
 1271 set. However, for consistency with our solver framework—which uniformly minimizes objective
 1272 values—we convert it into an equivalent minimization problem by taking the negative of the original
 1273 objective.

$$\begin{aligned}
 \min \quad & - \sum_{v \in \mathcal{V}} x_v \\
 \text{s.t.} \quad & x_u + x_v \leq 1, \quad \forall (u, v) \in \mathcal{E}, \\
 & x_v \in \{0, 1\}, \quad \forall v \in \mathcal{V}.
 \end{aligned} \tag{4}$$

1274 Here, $x_v = 1$ indicates that node v is selected into the independent set. The constraint ensures that
 1275 no two adjacent nodes are selected simultaneously. By minimizing the negated objective, the solver
 1276 effectively finds a maximum independent set while remaining compatible with our unified evaluation
 1277 setup.

1278 **Mixed 0-1 Knapsack Set (MKS)** [39]: This is a hybrid variant of the classical set cover and
 1279 knapsack problems. Each item must be covered by at least one set, and each set may be either binary
 1280 (selected/not) or continuous (fractionally selected). This formulation models resource allocation
 1281 scenarios with mixed decision types.

1282 Given N sets and M items, define x_i as the selection variable for set i , where $x_i \in \{0, 1\}$ for binary
 1283 sets and $x_i \in [0, 1]$ for continuous ones. Each item j must be covered by at least one selected set:

Table 3: Problem sizes (number of variables and constraints) across training, medium, and large scales.

Problem	Scale	#Variables	#Constraints
SC (Minimize)	SC ₀	40,000	40,000
	SC ₁	200,000	200,000
	SC ₂	2,000,000	2,000,000
MVC (Minimize)	MVC ₀	20,000	60,000
	MVC ₁	100,000	300,000
	MVC ₂	1,000,000	3,000,000
MIS (Maximize)	MIS ₀	20,000	60,000
	MIS ₁	100,000	300,000
	MIS ₂	1,000,000	3,000,000
MKS (Minimize)	MKS ₀	20,000	20,000
	MKS ₁	200,000	200,000
	MKS ₂	2,000,000	2,000,000

$$\begin{aligned}
\min \quad & \sum_{i=1}^N c_i x_i \\
\text{s.t.} \quad & \sum_{i:j \in S_i} x_i \geq 1, \quad \forall j \in \{1, \dots, M\}, \\
& x_i \in \{0, 1\} \text{ or } [0, 1], \quad \forall i \in \{1, \dots, N\}.
\end{aligned} \tag{5}$$

In our data generation process, each item appears in exactly four sets. Half of the sets are assigned binary variables, and the other half continuous variables. Coefficients c_i are randomly sampled from a uniform distribution in $[0.1, 1.0]$.

C.2 Problem Scale and Objective Statistics

We evaluate LEGO on four representative NP-hard MILP benchmarks: Set Covering (SC) [36], Minimum Vertex Cover (MVC) [37], Maximum Independent Set (MIS) [38], and Mixed 0-1 Knapsack Set (MKS) [39]. These problems are widely studied in combinatorial optimization and exhibit diverse structural properties, making them ideal for evaluating general-purpose MILP solvers.

For each problem, we construct instances at three different scales to comprehensively assess the generalization and scalability of our approach:

- **Training Scale** (₀): Small-to-moderate sized MILPs used for supervised training of neural components. These instances are designed to be efficiently solvable, enabling the collection of high-quality labels and model pretraining. Each problem includes **20 training instances** at this scale.
- **Medium Scale** (₁): Realistic problem sizes with approximately 100K decision variables and constraints. These serve as the primary benchmark for evaluating solution quality under strict runtime constraints.
- **Large Scale** (₂): Challenging MILPs with up to 1M–3M variables and constraints. These instances test the scalability and robustness of LEGO, especially in scenarios beyond the training distribution.

We ensure that the training and test sets are disjoint across all scales. The use of scale-separated evaluation allows us to analyze both in-distribution performance (on medium-scale problems) and out-of-distribution generalization (on large-scale problems). Detailed statistics on the number of variables and constraints for each problem and scale are summarized in Table 3.

Experimental Protocol Overview. We conduct comprehensive experiments to evaluate the effectiveness, adaptability, and scalability of LEGO in solving mixed-integer linear programs (MILPs) through

Table 4: Time budget (in seconds) used for evaluating each problem scale.

Problem Type	Medium Scale Time (s)	Large Scale Time (s)
Set Covering (SC)	600	3500
Vertex Cover (MVC)	600	3500
Independent Set (MIS)	600	3500
Mixed Knapsack Set (MKS)	4000	8000

Table 5: Objective values achieved by Light-MILPopt under the time budgets in Table 4. These serve as the reference target for LEGO.

Problem	Scale	Objective (Light)	Task Type
SC	SC ₁	16528.1	Min
SC	SC ₂	164154.3	Min
MVC	MVC ₁	27548.1	Min
MVC	MVC ₂	278557.6	Min
MKS	MKS ₁	20589.2	Min
MKS	MKS ₂	208803.5	Min
MIS	MIS ₁	−22900.1	Min
MIS	MIS ₂	−228611.5	Min

Predict-and-Search pipeline construction. The evaluation spans a diverse set of NP-hard MILP problems, including both synthetic benchmarks and real-world-like large-scale instances. LEGO is compared against classical solvers (Gurobi, SCIP) and representative learning-based baselines (e.g., Light-MILPopt), under standardized and reproducible experimental protocols.

To ensure a fair and meaningful comparison, we adopt two complementary evaluation schemes:

- **Fixed-Time Evaluation:** All methods are evaluated under identical runtime budgets (see Table 4). The goal is to assess solution quality within limited time, simulating real-time or resource-constrained settings.
- **Fixed-Quality Evaluation:** All methods are run until they reach a predefined objective value target (see Table 5). This allows comparison of time efficiency in achieving high-quality solutions.

Time Budget Settings. The time budgets for fixed-time evaluation are selected based on the scale and complexity of each MILP class. Table 4 summarizes the test-time runtime limits assigned to each problem scale. For example, MKS is more difficult and therefore receives longer budgets compared to SC or MVC.

Target Performance Reference. To define target objective values for fixed-quality evaluation, we use the best solutions obtained by **Light-MILPopt** [26] under fixed time budgets. These values are reported in Table 5 and serve as reference points to test whether other methods—including LEGO—can match or surpass this strong baseline.

Convergence and Budget Justification. The time budgets in Table 4 are carefully selected to balance computational feasibility and solution quality. They are long enough to allow all tested methods—including classical solvers and learning-based baselines—to converge or reach stable performance within the allocated time.

This is confirmed by the convergence plots shown in Figure 3 of the main text, where each method demonstrates near-converged objective values before the runtime limit is reached. These results support the validity of our fixed-time evaluation protocol and ensure that the comparisons reflect true solution quality rather than premature termination.

C.3 Hyperparameter Search Design

To ensure the constructed solver pipelines generalize across instance scales and problem types, LEGO incorporates a dedicated *Hyperparameter Search* stage to jointly tune component configurations

1340 and internal parameters. This process is critical for enhancing solution quality while maintaining
1341 size-transferability.

1342 **Size-Transferable Parameter Tuning.** As described in Appendix B.3, LEGO performs component
1343 selection and parameter tuning primarily on small-scale training instances. To address the discrepancy
1344 between training and deployment scales, we adopt a *Size-transferable Parameter Tuning* strategy.
1345 Specifically:

- 1346 • If training data includes instances with comparable size to test-time instances, tuning is
1347 conducted directly on these larger examples.
- 1348 • If only small-scale data is available, we generate structurally similar but larger synthetic
1349 MILPs using a controllable instance generator based on MILP-retrieval [21]. This ensures
1350 that tuning remains effective under limited data regimes.

1351 **Search Strategy and Budget.** We adopt *Bayesian Optimization* to efficiently explore the hyperpa-
1352 rameter space for each pipeline candidate. For each of the top- K configurations (ranked by initial
1353 performance), we search for the best parameter setting that maximizes solution quality under time
1354 constraints.

1355 To reduce search overhead, we initialize the optimization from a set of **expert-designed default**
1356 **parameters**, derived from prior experience and heuristics for each component. This warm-start
1357 significantly accelerates convergence.

1358 The number of search iterations is fixed and scale-aware:

- 1359 • **Medium-scale tuning:** 15 rounds of parameter search per candidate;
- 1360 • **Large-scale tuning:** 5 rounds of parameter search per candidate (due to higher cost).

1361 **Focus on Search Parameters.** Due to the high computational cost of retraining neural predictors
1362 whenever Predict-stage parameters change, we prioritize tuning parameters in the **Search stage**.
1363 These include:

- 1364 • Neighborhood size (for LNS/NALNS);
- 1365 • Variable block selection criteria (e.g., top- k integrality gap for LIH/MIH);
- 1366 • Subproblem timeout or iteration limits;
- 1367 • Warm-start acceptance thresholds for solver-based integration.

1368 By concentrating search on the most impactful and computationally tractable components, LEGO
1369 achieves a balance between tuning efficiency and performance gain.

1370 **Candidate Evaluation.** Each tuned configuration is evaluated using the *LLM-enhanced Performance*
1371 *Evaluation* module, which integrates numerical quality and structural reasoning. Final ranking is
1372 conducted via *Hypervolume-Based Component Candidates Comparison*, ensuring robustness across
1373 diverse instances. The top-ranked configuration is deployed as the final solver pipeline.

1374 C.4 Runtime and Cost Analysis

1375 We provide a detailed breakdown of the runtime and resource cost involved in training, component
1376 selection, parameter tuning, and final evaluation of LEGO. All experiments are conducted on a
1377 single workstation equipped with modern GPUs and CPUs, and the reported timing reflects realistic
1378 end-to-end usage.

1379 **Component Selection on Small-Scale Instances.** For each problem type, LEGO first performs
1380 component selection on small-scale training data. This process consists of three key steps:

- 1381 • **Predict Module Training:** We evaluate 4 types of graph embedding strategies combined
1382 with 2 types of GNN architectures, yielding 8 neural predictor variants in total. Each model
1383 is trained for 30 epochs on 20 training instances. On average, each model requires about 30
1384 minutes to converge, leading to a total of **4 hours** for training all predictors.

Algorithm 14 Instance Generator for Maximum Independent Set (MIS)

```
1: Input: Number of nodes  $n$ , graph type  $g \in \{\text{ER}, \text{BA}\}$ , edge probability  $p$ , affinity  $a$ 
2: Output: MILP instance for MIS
3: function GENERATEGRAPH( $n, g, p, a$ )
4:   if  $g = \text{ER}$  then
5:     Generate Erdős–Rényi graph  $G(n, p)$ 
6:   else if  $g = \text{BA}$  then
7:     Generate Barabási–Albert graph with affinity  $a$ 
8:   end if
9:   Compute degree of each node
10:  Return graph  $G$  with node list and edge set
11: end function
12: function CLIQUEPARTITION( $G$ )
13:  Initialize node list sorted by decreasing degree
14:  while unvisited nodes remain do
15:    Select node  $v$  with highest degree
16:    Initialize clique  $C = \{v\}$ 
17:    for neighbor  $u$  of  $v$  (by degree) do
18:      if  $u$  connected to all nodes in  $C$  then
19:         $C \leftarrow C \cup \{u\}$ 
20:      end if
21:    end for
22:    Mark nodes in  $C$  as visited
23:    Add  $C$  to clique list
24:  end while
25:  Return list of cliques
26: end function
27: function BUILDMISMODEL( $G$ , clique list)
28:  Create binary variable  $x_v$  for each node  $v$ 
29:  Create auxiliary binary variable  $y_v$  for critical nodes
30:  for clique  $C$  in clique list do
31:    Add constraint:  $\sum_{v \in C} x_v \leq 1$ 
32:  end for
33:  for critical node  $v$  do
34:    Add indicator constraint:  $\sum_{u \in \text{Nbr}(v)} x_u \leq (1 - y_v) \cdot |\text{Nbr}(v)|$ 
35:  end for
36:  Objective: maximize  $\sum x_v + \sum y_v$ 
37:  Return MILP model
38: end function
```

- 1385 • **Component Combination Evaluation:** After predictor training, we use a high-performance
1386 solver (e.g., Gurobi) to compute near-optimal lower bounds for each instance by running it
1387 for 3000 seconds. We then evaluate 336 candidate pipeline combinations, derived from:
- 1388 – 4 embedding types $\times$ 4 predictors $\times$ 3 repair strategies $\times$ 7 search modules.
- 1389 After removing invalid or redundant combinations (e.g., pipelines that apply repair after
1390 Gurobi solutions), we are left with 189 meaningful configurations. Each configuration is
1391 tested for 100 seconds, totaling approximately **18,900 seconds** (5.25 hours). Combined with
1392 the solver warm-up time, this stage takes around **6 hours** in total.
- 1393 • **Candidate Selection:** All configurations are evaluated using the previously described
1394 LLM-enhanced metrics. Top- K candidates are selected using non-dominated sorting and
1395 hypervolume contribution analysis. This step is computationally lightweight, requiring only
1396 a few minutes.
- 1397 **Hyperparameter Search for Top Candidates.** After identifying the top- K component combinations
1398 (we set $K = 3$ in practice), we perform parameter tuning for each. Using the Bayesian Optimization
1399 framework described earlier, each candidate undergoes 5 hours of search. This results in a total of **15**
1400 **hours** for hyperparameter tuning.

Algorithm 15 Instance Generator for Minimum Vertex Cover (MVC)

```
1: Input: Number of stations  $m$ , number of channels  $n$ 
2: Parameters: Demand interval  $[d_{\min}, d_{\max}]$ , capacity interval  $[c_{\min}, c_{\max}]$ , fixed cost bounds,
   bandwidth bounds
3: Output: MILP instance for MVC-style resource allocation
4: function GENERATEINSTANCE( $m, n$ )
5:   Sample demand  $d_i \sim \mathcal{U}(d_{\min}, d_{\max})$  for each channel  $i$ 
6:   Sample capacity  $c_j \sim \mathcal{U}(c_{\min}, c_{\max})$  for each station  $j$ 
7:   Sample fixed cost  $f_j$  and bandwidth  $b_j$  for each station  $j$ 
8:   Return  $\{d_i\}, \{c_j\}, \{f_j\}, \{b_j\}$ 
9: end function
10: function BUILDMVCMODEL( $\{d_i\}, \{c_j\}, \{f_j\}, \{b_j\}$ )
11:   Create binary variable  $x_j$  for each station  $j$  (whether activated)
12:   Create continuous variable  $y_{i,j}$  for bandwidth from station  $j$  to channel  $i$ 
13:   Objective: Minimize  $\sum_j f_j \cdot x_j$ 
14:   for each channel  $i$  do
15:     Add constraint:  $\sum_j y_{i,j} \geq d_i$  ▷ demand satisfaction
16:   end for
17:   for each station  $j$  do
18:     Add constraint:  $\sum_i y_{i,j} \leq b_j \cdot x_j$  ▷ bandwidth capacity
19:   end for
20:   Return MILP model
21: end function
```

1401 **Final Evaluation on Medium and Large Instances.** Once the optimal pipeline is selected, we
1402 evaluate its performance on both medium-scale and large-scale test sets. Each configuration is tested
1403 five times (to account for randomness in warm-starts and LNS), under the following runtime budgets:

- 1404 • **Medium-scale:** 4000 seconds $\times$ 5 runs = **20,000 seconds $\approx$ 5.5 hours**
- 1405 • **Large-scale:** 8000 seconds $\times$ 5 runs = **40,000 seconds $\approx$ 11 hours**

1406 **Total Runtime Summary.** Summing across all stages, the total wall-clock time for training, compo-
1407 nent selection, tuning, and evaluation is as follows:

- 1408 • **Training + Selection + Tuning:** 4 hours (training) + 6 hours (evaluation) + 15 hours (tuning)
1409 = **25 hours**
- 1410 • **Final Testing:** 5.5 hours (medium) + 11 hours (large) = **16.5 hours**

1411 This results in a total runtime of approximately **41.5 hours** per problem type. We note that most
1412 stages (e.g., embedding training and component evaluation) are fully parallelizable across different
1413 problem instances or configurations, and can be significantly accelerated in distributed environments.

1414 D Additional Results and Analysis

1415 To complement the main empirical findings, we present a set of additional experiments and analyses
1416 that provide deeper insights into the design, effectiveness, and robustness of LEGO. These results
1417 are essential for understanding the rationale behind key design choices and further validating the
1418 generalization capabilities of the proposed framework.

1419 Specifically, we include the following components:

- 1420 • A detailed evaluation of the MILP-Retrieval module applied to four benchmark problem
1421 types;
- 1422 • Full component ranking results and final candidate selections (including hyperparameters)
1423 for both medium and large-scale instances;
- 1424 • Ablation studies that examine the impact of each module in the evaluation and selection
1425 process, including metric design, ranking method, and parameter tuning;

Algorithm 16 Instance Generator for Set Covering (SC)

```
1: Input: Number of constraints  $m$ , number of variables  $n$ , density  $\rho$ 
2: Parameters: Cost upper bound  $c_{\max}$ , budget  $B$ , average service quality  $\bar{q}$ , quality weight  $\lambda$ ,
   quality threshold  $\theta$ 
3: Output: MILP instance for generalized Set Covering
4: function GENERATECOVERMATRIX( $m, n, \rho$ )
5:   Compute number of non-zeros:  $nnz = \rho \cdot m \cdot n$ 
6:   Randomly assign  $nnz$  binary entries to a sparse matrix  $A \in \{0, 1\}^{m \times n}$  s.t. each column
   covers at least 2 rows
7:   Return matrix  $A$  in CSR format
8: end function
9: function GENERATECOEFFICIENTS( $n$ )
10:  Sample cost vector  $c_j \sim \mathcal{U}(1, c_{\max})$ 
11:  Sample quality scores  $q_j \sim \mathcal{N}(\bar{q}, \sigma)$ 
12:  Return  $c, q$ 
13: end function
14: function BUILDSCMODEL( $A, c, q$ )
15:  Create binary decision variable  $x_j$  for each column  $j$ 
16:  Create binary auxiliary variable  $z_j$  for service quality modeling
17:  for each row  $i$  in  $A$  do
18:    Add constraint:  $\sum_{j \in \text{supp}(i)} x_j \geq 1$ 
19:  end for
20:  Add budget constraint:  $\sum_j c_j \cdot x_j \leq B$ 
21:  Add service quality constraint:  $\sum_j z_j \geq Q_{\min}$ 
22:  for each  $j$  do
23:    if  $q_j > \theta$  then
24:      Add logic constraint:  $x_j \leq \sum_{k \in \text{Nbr}(j)} x_k$ 
25:    end if
26:  end for
27:  Objective: Minimize  $\sum_j c_j x_j - \lambda \sum_j q_j z_j$ 
28:  Return MILP model
29: end function
```

1426 • Statistical robustness analysis through error bar comparisons between LEGO and Light-
1427 MILPopt.

1428 These additional results reinforce the reliability of LEGO across a wide range of MILP scenarios,
1429 and highlight the flexibility of its modular design.

1430 D.1 MILP-Retrieval Results on Four Benchmarks

1431 To enable size-transferable learning and parameter tuning, LEGO leverages a MILP-Retrieval module
1432 that synthesizes problem instances structurally aligned with a given target MILP. This is particularly
1433 useful when only small-scale training data is available. The retrieved generators are capable of
1434 producing synthetic yet realistic MILPs that preserve the combinatorial structure and solver behavior
1435 of the original problems.

1436 Below, we present the retrieved generation code for each of the four benchmark problems, along with
1437 a brief analysis of their structural similarity to the original datasets.

1438 **(1) Maximum Independent Set (MIS).** As shown in Algorithm 14, the generator constructs graphs
1439 using either Erdős–Rényi [48] or Barabási–Albert [49] models, and enforces clique-based mutual
1440 exclusion constraints. Additionally, it introduces auxiliary binary variables to simulate critical node
1441 constraints. The inequality structure and variable sparsity closely resemble our real MIS benchmarks.
1442 The use of greedy clique partitioning produces realistic conflict structures.

1443 **(2) Minimum Vertex Cover (MVC).** As shown in Algorithm 15, the retrieved generator simulates
1444 a resource allocation problem where nodes represent stations and edges represent communication

Algorithm 17 Instance Generator for Mixed Knapsack Set (MKS)

```
1: Input: Node range  $[n_{\min}, n_{\max}]$ , edge probability  $p$ , edge removal rate  $\alpha$ , budget  $B$ 
2: Output: MILP instance with mixed 0-1 decision variables
3: function GENERATEGRAPH( $n_{\min}, n_{\max}, p$ )
4:   Sample node count  $n \sim \mathcal{U}(n_{\min}, n_{\max})$ 
5:   Generate random graph  $G(n, p)$ 
6:   Assign to each node:
   • Population  $P_v \sim \mathcal{U}(100, 1000)$ 
   • Setup cost  $C_v \sim \mathcal{U}(200, 1000)$ 
7:   Assign to each edge  $(u, v)$ :
8:     Connection cost  $C_{uv} = \frac{C_u + C_v}{3}$ 
9:   Return graph  $G$ 
10: end function
11: function REMOVEEDGES( $G, \alpha$ )
12:   Randomly sample a subset  $E_2 \subseteq E(G)$  such that  $|E_2| \approx \alpha \cdot |E|$ 
13:   Return removable edge set  $E_2$ 
14: end function
15: function BUILDMKSMODEL( $G, E_2, B$ )
16:   Create binary variable  $S_v$  for settlement at node  $v$ 
17:   Create binary variable  $F_v$  for funding at node  $v$ 
18:   Create binary variable  $C_{uv}$  for activating edge  $(u, v) \in E_2$ 
19:   Objective: Maximize
      
$$\sum_v P_v \cdot S_v - \sum_v C_v \cdot F_v - \sum_{(u,v) \in E_2} C_{uv} \cdot C_{uv}$$

20:   Add budget constraint:
      
$$\sum_v C_v \cdot S_v \leq B$$

21:   for each  $(u, v) \in E_2$  do
22:     Add constraint:  $S_u + S_v - C_{uv} \leq 1$ 
23:   end for
24:   for each node  $v$  do
25:     Add constraint:  $F_v \geq S_v$ 
26:   end for
27:   Return MILP model
28: end function
```

1445 demands. The MILP formulation captures coverage and capacity constraints, effectively modeling
1446 vertex cover behavior. The allocation and bandwidth balance mimic the edge-node incidence structure
1447 of MVC, while the objective function retains the minimization of node activations.

1448 **(3) Set Covering (SC).** As shown in Algorithm 16, the SC generator constructs a large-scale coverage
1449 matrix using sparse random patterns, while enforcing additional service quality, budget, and logic
1450 constraints. The coverage matrix is structurally similar to classical SC formulations, with sparse
1451 binary entries ensuring that each constraint is covered by at least one variable. The inclusion of
1452 correlated cost-quality variables enhances the realism of the generated problem.

1453 **(4) Mixed 0-1 Knapsack Set (MKS).** As shown in Algorithm 17, this generator creates a utility-driven
1454 edge activation problem over random graphs, where decisions involve both binary and continuous
1455 variables. It incorporates a knapsack-style budget constraint and edge activation dependencies,
1456 reflecting the hybrid combinatorial and continuous nature of MKS. The instance structure exhibits
1457 similar sparsity and variable types to the original benchmark.

1458 **Summary.** These retrieved generators are automatically selected using the MILP-Retrieval
1459 method [21], which matches the structure and difficulty of target problems. They provide a re-
1460 liable source of large-scale, synthetic MILPs that maintain alignment with real-world problem

Table 6: Top-3 component candidates selected under full 7-dimensional evaluation. Larger hypervolume contribution indicates better trade-off across objectives.

Problem	Candidate Pipeline	HV Contribution
MIS	Tripartite GCN + Neighborhood Repair + NALNS	59355.0
	Bipartite GCN + Neighborhood Repair + NALNS	43901.0
	Tripartite GCN + Adaptive Radius + ACP	28827.0
SC	Bipartite GCN + Adaptive Threshold + ACP	159853.2
	default + Gurobi + default + ACP	70877.8
	Bipartite GAT + Neighborhood Repair + ACP	37882.0
MKS	Bipartite GCN + Neighborhood Repair + ACP	177649.7
	Bipartite GCN + Adaptive Threshold + ACP	61835.7
	Tripartite GCN + Neighborhood Repair + ACP	46653.2
MVC	Bipartite GCN + Neighborhood Repair + ACP	106955.0
	Bipartite GAT + Neighborhood Repair + ACP	40982.0
	Bipartite GAT + Adaptive Threshold + ACP	31708.0

Table 7: Final selected component pipelines for each benchmark after full tuning. “Real” and “Gen” refer to real and synthetic datasets, respectively.

Problem	Real	Gen
<i>Medium-scale Instances</i>		
MIS	Tripartite GCN + Neighborhood Repair	Tripartite GCN + Adaptive Radius + ACP
SC	Bipartite GAT + Neighborhood Repair + ACP	Bipartite GAT + Neighborhood Repair + ACP
MVC	Bipartite GCN + Neighborhood Repair + ACP	Bipartite GCN + Neighborhood Repair + ACP
MKS	Bipartite GAT + Adaptive Threshold + ACP	Bipartite GAT + Adaptive Threshold + ACP
<i>Hard-scale Instances</i>		
MIS	Tripartite GCN + Neighborhood Repair	Tripartite GCN + Neighborhood Repair
SC	Bipartite GCN + Adaptive Threshold + ACP	Bipartite GCN + Adaptive Threshold + ACP
MVC	Bipartite GCN + Neighborhood Repair + ACP	Bipartite GCN + Neighborhood Repair + ACP
MKS	Bipartite GCN + Neighborhood Repair + ACP	Bipartite GCN + Neighborhood Repair + ACP

distributions. This enables LEGO to perform effective parameter tuning even when direct access to large labeled MILPs is limited.

D.2 Component Ranking and Final Selection

Based on the 7-dimensional evaluation metrics introduced in Appendix B.1, we rank all candidate component combinations using a hypervolume-based contribution analysis. This strategy allows us to perform multi-objective selection that accounts for trade-offs across solution quality, robustness, convergence, and LLM-based assessments.

Top-3 Candidates by Hypervolume Contribution. Table 6 shows the top-3 ranked component pipelines for each benchmark problem, along with their hypervolume contributions. The results reveal clear distinctions between pipelines in terms of generalization potential.

Final Component Selection after Parameter Tuning. After hyperparameter tuning (as described in Appendix B.1), we select the best-performing pipeline for each benchmark under four settings: medium-scale and large-scale, with both real (Real) and synthetic (Gen) instances.

Table 7 summarizes the final selected pipelines.

Analysis. We observe that certain component configurations consistently emerge across data regimes. For instance, the *GCN + Neighborhood Repair* combination proves highly effective for MIS and MVC, while *GAT + Adaptive Threshold* dominates under MKS. Notably, the adaptive search modules (e.g., ACP, NALNS) are frequently selected in large-scale settings, indicating their value in handling scalability.

These results validate that our hypervolume-guided ranking combined with size-transferable parameter tuning leads to stable and generalizable solver designs.

Table 8: Ablation on the use of evaluation metrics. “without-Metrics” variants use only primal gap for component ranking. LEGO-Real and LEGO-Gen refer to our full versions using multi-dimensional evaluation (numerical + LLM feedback). Lower is better for SC, MVC, MIKS; higher is better for MIS.

	SC ₁	MVC ₁	MIKS ₁	MIS ₁
Gurobi	24313.0	27925.8	34285.3	-21966.7
LEFO-Real (w/o Metrics)	16892.5	27707.8	34987.6	-22212.5
LEFO-Gen (w/o Metrics)	18409.3	27707.8	34987.6	-22212.5
LEGO-Real (Ours)	16108.6↑	26675.4↑	19957.7↑	-23085.9↑
LEGO-Gen (Ours)	16183.3↑	26709.9↑	20063.3↑	-23204.0↑

D.3 Ablation Studies

To investigate the contribution of each core component in LEGO, we conduct a series of ablation studies targeting its most critical design choices. In particular, we evaluate the impact of the following modules:

- **LLM-enhanced Performance Evaluation**, which integrates human knowledge and numerical statistics to guide component selection;
- **Hypervolume-guided Component Selection**, which provides a multi-objective ranking mechanism robust to trade-offs across diverse instances;
- **Size-transferable Parameter Tuning**, which ensures that the selected configurations generalize effectively from small-scale to large-scale settings.

For each of these modules, we design controlled experiments that isolate the corresponding functionality and compare LEGO with its ablated variant. The results demonstrate that removing any of these components leads to significant performance degradation, confirming their essential role in achieving LEGO’s overall effectiveness and generalization ability.

D.3.1 Effect of Evaluation Metrics

To evaluate the importance of our multi-dimensional evaluation design, we conduct an ablation study where only the primal gap is used as the ranking criterion during component selection. In this setting, the LLM-enhanced evaluation module is disabled, and all pipeline candidates are scored and ranked solely by their average primal gap on small-scale instances.

Table 8 shows the performance of LEGO variants with and without full evaluation metrics. While the “w/o Metrics” versions still outperform Gurobi on some problems, the complete LEGO (both Real and Gen) achieves consistently better performance across all benchmarks. This demonstrates that using only gap as a ranking signal is insufficient for robust solver design.

Top-3 Candidates under Gap-Only Evaluation. To further understand the limitations of gap-only evaluation, we analyze the top-3 ranked component configurations selected under this setting. Table 9 lists the top pipelines (with their gap values) for each benchmark:

Observations. We make several key observations from these gap-only selections:

- For **MIS** and **MVC**, the top-ranked pipelines all use Gurobi as the search module, which performs very well on small-scale instances. However, these configurations fail to generalize to large-scale problems, as Gurobi often struggles without warm-start heuristics in combinatorial-heavy domains.
- In **SC**, the selected candidates use LNS-type search modules, which are more scalable. As a result, even gap-only ranking yields reasonably good performance—though still inferior to LEGO’s final choice.
- For **MIKS**, the top candidate is the default Gurobi configuration without any learning components. While this achieves a low gap on small instances, its lack of structure-aware search limits its effectiveness in larger settings.

Table 9: Top-3 candidates ranked by primal gap only. Lower gap is better. “default” indicates the absence of that module (e.g., no repair or no predictor).

Problem	Embed	Predict	Repair	Search	Gap
MIS	Bipartite	GCN	Neighborhood Repair	Gurobi	0.07265
	Tripartite	GCN	Neighborhood Repair	ACP	0.07318
	Bipartite	GCN	Neighborhood Repair	ACP	0.07319
SC	Tripartite	GCN	Adaptive Radius	LNS	0.06435
	default	SCIP	default	LNS	0.06462
	Tripartite	GCN	Adaptive Radius	LNS	0.06651
MVC	Bipartite	GCN	Neighborhood Repair	Gurobi	0.07030
	Bipartite	GAT	Adaptive Threshold	ACP	0.07125
	Tripartite	GCN	Neighborhood Repair	ACP	0.07134
MIKS	default	Gurobi	default	LIH	0.02686
	Bipartite	GCN	Adaptive Threshold	LIH	0.02898
	Tripartite	GCN	Neighborhood Repair	LIH	0.03183

Table 10: Ablation on hypervolume-based component ranking. “without-Hypervolume” variants use mean score across metrics to rank candidates instead of hypervolume contribution. LEGO-Real and LEGO-Gen use full hypervolume-guided selection. Lower is better for SC, MVC, MIKS; higher is better for MIS.

	SC ₁	MVC ₁	MIKS ₁	MIS ₁
Gurobi	24313.0	27925.8	34285.3	-21966.7
LEFO-Real (w/o Hypervolume)	17919.5	26701.1	20567.0	-23084.1
LEFO-Gen (w/o Hypervolume)	17201.3	27321.7	20716.1	-23159.8
LEGO-Real (Ours)	16108.6↑	26675.4↑	19957.7↑	-23085.9↑
LEGO-Gen (Ours)	16183.3↑	26709.9↑	20063.3↑	-23204.0↑

Conclusion. These results highlight the limitations of using primal gap as the sole evaluation metric. While gap is a useful baseline indicator of solver performance, it fails to account for structural robustness, convergence behavior, and the potential of search components under tight time budgets. Our full evaluation strategy—integrating both numerical and LLM-guided feedback—enables LEGO to more reliably identify pipelines that generalize across instance scales.

D.3.2 Effect of Hypervolume-Based Ranking

To validate the importance of our hypervolume-guided component selection strategy, we conduct an ablation in which the hypervolume contribution metric is replaced with a naive mean-based ranking. In this baseline, each candidate pipeline is scored by the average of its 7-dimensional evaluation vector, treating all objectives as equally weighted and linearly comparable.

As shown in Table 10, both LEGO-Real and LEGO-Gen variants suffer consistent performance degradation when hypervolume is disabled. The decline is especially pronounced on SC and MIKS, where the final objective values deteriorate by over 1,000 units compared to the full version. This highlights the fragility of mean-based scoring in multi-objective optimization.

Why does hypervolume ranking outperform mean-based scoring? We attribute the performance gap to two fundamental issues:

1. **Loss of Pareto-awareness.** Mean-based ranking assumes a flat objective space where all dimensions are additive and equally important. However, in practice, certain objectives—such as solution optimality or convergence stability—may dominate others in downstream performance. Averaging across dimensions can mask these critical signals and favor pipelines that are uniformly mediocre rather than strategically strong. Worse still, mean-based ranking is blind to Pareto dominance: it may select candidates that are strictly dominated by others in all objectives, thereby wasting valuable candidate slots and reducing the diversity of the search frontier.
2. **Disrupted tuning dynamics.** Hypervolume-based ranking tends to select candidates that lie on the Pareto frontier, covering diverse trade-offs and behaviors. This diversity is essential for subsequent parameter tuning, as it increases the likelihood of discovering

Table 11: Ablation on parameter transfer. “without-Transfer” disables size-transferable tuning, using small-scale parameters directly on large-scale tasks. LEGO-Real/Gen apply full transfer-based tuning. Lower is better for SC, MVC, MIKS; higher is better for MIS.

	SC ₁	MVC ₁	MIKS ₁	MIS ₁
Gurobi	24313.0	27925.8	34285.3	-21966.7
LEFO-Real (w/o Transfer)	18730.1	28528.6	22119.3	-18539.2
LEGO-Real (Ours)	16108.6↑	26675.4↑	19957.7↑	-23085.9↑
LEGO-Gen (Ours)	16183.3↑	26709.9↑	20063.3↑	-23204.0↑

complementary local optima. In contrast, mean-based ranking often yields pipelines with similar performance profiles, leading to redundant tuning paths. With limited tuning budgets, this lack of behavioral diversity makes the system more prone to premature convergence and suboptimal generalization.

Empirical observations. In our experiments, we observe that mean-based rankings frequently elevate pipelines that are “average” in every metric but not outstanding in any. For example, pipelines with slightly-above-average scores in runtime, quality, and LLM feedback may outperform a pipeline that is significantly better in solution quality but slightly worse in interpretability. This leads to the exclusion of potentially high-impact configurations.

Conclusion. These findings confirm that hypervolume-based ranking is essential for robust multi-objective solver design. It not only ensures that selected candidates span a representative and non-dominated frontier, but also supports downstream tuning by preserving behavioral diversity. By doing so, it substantially enhances the generalization ability of LEGO across both compositional and scale shifts.

D.3.3 Effect of Parameter Transfer

To assess the effectiveness of our size-transferable parameter tuning strategy, we conduct an ablation where no adaptation is performed for large-scale instances. Specifically, we take the component pipelines and parameters selected on small-scale data and apply them directly to large-scale benchmarks, without any parameter refinement or scale-specific adjustment.

As shown in Table 11, this “without-Transfer” setting leads to clear performance degradation across all benchmarks. For instance, performance on MIS drops from -23085.9 to -18539.2 , and similar regressions are observed in SC and MIKS. This confirms that directly using small-scale configurations on large problems is suboptimal.

Why does transfer matter? We identify two main reasons:

1. **Impact on Component Effectiveness.** While component pipelines are selected based on small-scale performance, our earlier ablation on evaluation metrics (Appendix B.2.2) already showed that some components favored by gap or mean-based scores may not generalize to large instances. For example, Gurobi-based search modules often appear strong in small-scale settings but perform poorly in large-scale scenarios due to scalability issues. Without transfer, such pipelines may be retained despite their lack of robustness.
2. **Impact on Parameter Search.** Even for well-chosen components, the optimal parameter settings often differ significantly between small and large instances. For example, a perturbation radius or neighborhood size that is effective on a 100-node graph may be too narrow or too wide for a 1000-node instance. Similarly, search time budgets that suffice for fast-converging small instances may be inadequate for larger problems. Without parameter transfer, the solver is often either over-exploring or under-utilizing its time budget.

Conclusion. The size-transferable tuning strategy in LEGO plays a dual role: it not only adjusts hyperparameters to match the scale of the problem, but also allows re-evaluation and refinement of component effectiveness under realistic conditions. This ensures that the final configuration is both structurally robust and computationally efficient at large scales.

Table 12: Standard deviation of final objective values over 5 independent runs on medium-scale benchmark instances. Lower is better.

	SC ₁	MVC ₁	MIKS ₁	MIS ₁
Light-MILPopt	1.82	1.36	0.32	0.21
LEGO-Real (Ours)	5.73	17.07	8.23	9.62
LEGO-Gen (Ours)	15.75	20.75	10.19	4.58

D.4 Error Bar and Robustness Analysis

To assess the robustness of LEGO, we evaluate the variability of its performance across repeated runs. Specifically, we conduct five independent runs for each solver configuration on medium-scale benchmark instances, and report the standard deviation (std) of the final objective values. The comparison is presented in Table 12.

Overall, LEGO exhibits strong robustness across all four benchmarks. Although its standard deviation is slightly higher than the Light-MILPopt baseline, the absolute differences remain small relative to the magnitude of the objective values (typically in the range of 10^4 – 10^5). For example, on the SC benchmark, LEGO-Real has a std of 5.73, while Light-MILPopt has a std of 1.82—both values are under 0.01% of the total objective scale.

Interpretation. Two key observations emerge:

- **Stochasticity vs. Performance.** LEGO introduces controlled stochasticity through its P&S (Predict-and-Search) architecture, which explains its relatively higher std. However, this variability is offset by significantly higher average performance compared to Light-MILPopt, as shown in earlier sections.
- **Stability at Scale.** Despite using learning-based components and adaptive search, the variance remains tightly bounded. This suggests that LEGO’s selection and tuning mechanisms are stable and repeatable, even under random initialization or heuristic sampling.

Conclusion. These results demonstrate that LEGO achieves a favorable trade-off between performance and stability. While its stochastic design introduces modest variance, the overall standard deviation remains low, and well within acceptable bounds for large-scale combinatorial optimization. This confirms the reliability of LEGO as a practical solver framework.

E Future Work Directions

While LEGO demonstrates strong performance and generalization across a variety of combinatorial problems, several important directions remain open for future exploration and improvement.

Robustness to Synthetic Data Generation. In scenarios where real-world or large-scale training data is limited, LEGO relies on synthetic instance generators to produce task-relevant data for training and evaluation. Although this strategy enables scalability, the final component selection and tuning outcomes can be sensitive to the quality and diversity of the generated instances. In future work, we aim to reduce this dependency by developing generator-agnostic training mechanisms or incorporating generator uncertainty into the component selection process.

Limits of Scale Generalization. Our current framework supports generalization across moderate scale shifts (e.g., from small to medium/large instances). However, we observe that the effectiveness of such transfer saturates beyond a certain point. In particular, components that perform well on small instances may become computationally prohibitive or less effective on significantly larger instances due to increased complexity or diminishing returns. A promising future direction is to develop hierarchical or scale-aware selection strategies that adapt the choice of components based on instance scale and available computational budgets.

Fully Automated Evaluation Metric Selection. While the current 7-dimensional evaluation vector integrates both numerical metrics and LLM-generated qualitative scores, the evaluation process still involves human-defined aggregation and interpretation. In future work, we plan to explore data-driven

approaches for automatically learning which evaluation dimensions most strongly correlate with downstream solver performance. This would enable LEGO to autonomously select and weight metrics per task, improving adaptability and reducing reliance on manual metric engineering.

By addressing these challenges, we aim to further enhance LEGO’s robustness, scalability, and autonomy, paving the way for more reliable and deployable learning-enhanced optimization systems.

References

- [1] Stefan Bock, Stefan Bomsdorf, Nils Boysen, and Michael Schneider. A survey on the traveling salesman problem and its variants in a warehousing context. *European Journal of Operational Research*, 2025.
- [2] Meng-Yu Huang, Ling-Ying Huang, Yu-Xing Zhong, Hui-Wen Yang, Xiao-Meng Chen, Wei Huo, Jia-Zheng Wang, Fan Zhang, Bo Bai, and Ling Shi. Milp acceleration: A survey from perspectives of simplex initialization and learning-based branch and bound. *Journal of the operations research society of china*, 2025.
- [3] Phuong Le. A survey on combinatorial optimization. *arXiv preprint arXiv:2409.00075*, 2024.
- [4] Per Kristian Lehre and Shishen Lin. No free lunch theorem and black-box complexity analysis for adversarial optimisation. *Advances in Neural Information Processing Systems*, 2024.
- [5] Xiaoqian Li and Kwan L Yeung. Traffic engineering in segment routing networks using milp. *IEEE Transactions on Network and Service Management*, 2020.
- [6] Carlos A Mendez and Jaime Cerdá. An milp framework for batch reactive scheduling with limited discrete resources. *Computers & Chemical Engineering*, 2004.
- [7] Raine Jokinen, Frank Pettersson, and Henrik Saxén. An milp model for optimization of a small-scale lng supply chain along a coastline. *Applied Energy*, 2015.
- [8] Qingyu Han, Linxin Yang, Qian Chen, Xiang Zhou, Dong Zhang, Akang Wang, Ruoyu Sun, and Xiaodong Luo. A gnn-guided predict-and-search framework for mixed-integer linear programming. In *The International Conference on Learning Representations*, 2023.
- [9] Taoan Huang, Aaron M Ferber, Arman Zharmagambetov, Yuandong Tian, and Bistra Dilkina. Contrastive predict-and-search for mixed integer linear programs. In *International Conference on Machine Learning*, 2024.
- [10] Frank Hutter, Holger H Hoos, Kevin Leyton-Brown, and Thomas Stützle. Paramils: An automatic algorithm configuration framework. *Journal of artificial intelligence research*, 2009.
- [11] Gellért Weisz, Andras Gyorgy, and Csaba Szepesvári. Leapsandbounds: A method for approximately optimal algorithm configuration. In *International Conference on Machine Learning*, 2018.
- [12] Lars Kotthoff. Algorithm selection for combinatorial search problems: A survey. In *Data Mining and Constraint Programming: Foundations of a Cross-Disciplinary Approach*. 2016.
- [13] Zeyuan Ma, Hongshu Guo, Yue-Jiao Gong, Jun Zhang, and Kay Chen Tan. Toward automated algorithm design: A survey and practical guide to meta-black-box-optimization. *arXiv preprint arXiv:2411.00625*, 2024.
- [14] Massimo Regona, Tan Yigitcanlar, Bo Xia, and Rita Yi Man Li. Opportunities and adoption challenges of ai in the construction industry: A prisma review. *Journal of open innovation: technology, market, and complexity*, 2022.
- [15] Marcus Tantakoun, Xiaodan Zhu, and Christian Muise. Llms as planning modelers: A survey for leveraging large language models to construct automated planning models. *arXiv preprint arXiv:2503.18971*, 2025.
- [16] Frank Hutter, Lars Kotthoff, and Marius Lindauer. Algorithm configuration: Challenges, methods and perspectives. Tutorial at IJCAI 2020, 2020. Accessed: 2025-05-11.

- [17] Tobias Achterberg. Constraint integer programming. 2007.
- [18] A. Schrijver. *Theory of Linear and Integer Programming*. John Wiley & Sons, 1998.
- [19] Jiayi Zhang, Chang Liu, Xijun Li, Hui-Ling Zhen, Mingxuan Yuan, Yawen Li, and Junchi Yan. A survey for solving mixed integer programming via machine learning. *Neurocomputing*, 2023.
- [20] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- [21] Tianxing Yang, Huigen Ye, and Hua Xu. Code retrieval for milp instance generation. *arXiv preprint arXiv:2505.11526*, 2025.
- [22] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. *Advances in neural information processing systems*, 2019.
- [23] Jian-Ya Ding, Chao Zhang, Lei Shen, Shengyin Li, Bing Wang, Yinghui Xu, and Le Song. Accelerating primal solution findings for mixed integer programs based on solution prediction. In *AAAI Conference on Artificial Intelligence*, 2020.
- [24] Ziang Chen, Jialin Liu, Xinshang Wang, and Wotao Yin. On representing mixed-integer linear programs by graph neural networks. In *International Conference on Learning Representations*, 2023.
- [25] Nicolas Sonnerat, Pengming Wang, Ira Ktena, Sergey Bartunov, and Vinod Nair. Learning a large neighborhood search algorithm for mixed integer programs. *arXiv preprint arXiv:2107.10201*, 2021.
- [26] Huigen Ye, Hua Xu, and Hongyan Wang. Light-milpopt: Solving large-scale mixed integer linear programs with lightweight optimizer and small-scale training dataset. In *International Conference on Learning Representations*, 2024.
- [27] Tobias Achterberg. What’s new in gurobi 9.0. *Webinar Talk url: <https://www.gurobi.com/wp-content/uploads/2019/12/Gurobi-90-Overview-Webinar-Slides-1.pdf>*, 2019.
- [28] Tobias Achterberg. Scip: Solving constraint integer programs. *Mathematical Programming Computation*, 2009.
- [29] Taehyun Yoon. Confidence threshold neural diving. *arXiv preprint arXiv:2202.07506*, 2022.
- [30] Huigen Ye, Hua Xu, Hongyan Wang, Chengming Wang, and Yu Jiang. Gnn & gbdt-guided fast optimizing framework for large-scale integer programming. In *International Conference on Machine Learning*, 2023.
- [31] Jialin Song, Yisong Yue, Bistra Dilikina, et al. A general large neighborhood search framework for solving integer linear programs. *Advances in Neural Information Processing Systems*, 2020.
- [32] Gregor Hendel. Adaptive large neighborhood search for mixed integer programming. *Mathematical Programming Computation*, 2022.
- [33] Vinod Nair, Mohammad Alizadeh, et al. Neural large neighborhood search. In *Learning Meets Combinatorial Algorithms at NeurIPS2020*, 2020.
- [34] Timo Berthold. *Primal heuristics for mixed integer programs*. PhD thesis, Zuse Institute Berlin (ZIB), 2006.
- [35] Huigen Ye, Hongyan Wang, Hua Xu, Chengming Wang, and Yu Jiang. Adaptive constraint partition based optimization framework for large-scale integer linear programming. In *AAAI Conference on Artificial Intelligence*, 2023.
- [36] Egon Balas and Manfred W Padberg. On the set-covering problem. *Operations Research*, 1972.
- [37] Samir Khuller. Algorithms column: The tertext cover problem. *ACM SIGACT News*, 2002.

- 1718 [38] Jan-Henrik Haunert and Alexander Wolff. Beyond maximum independent set: An extended
1719 integer programming formulation for point labeling. *ISPRS International Journal of Geo-*
1720 *Information*, 2017.
- 1721 [39] Alper Atamtürk. Cover and pack inequalities for (mixed) integer programming. *Annals of*
1722 *Operations Research*, 2005.
- 1723 [40] Kalyanmoy Deb, Samir Agrawal, Amrit Pratap, and Tanaka Meyarivan. A fast elitist non-
1724 dominated sorting genetic algorithm for multi-objective optimization: Nsga-ii. In *International*
1725 *conference on parallel problem solving from nature*, pages 849–858. Springer, 2000.
- 1726 [41] Karl Bringmann and Tobias Friedrich. An efficient algorithm for computing hypervolume
1727 contributions. *Evolutionary Computation*, 18(3):383–402, 2010.
- 1728 [42] Czesław Smutnicki, Jarosław Rudy, and Dominik Żelazny. Very fast non-dominated sorting.
1729 *Decision Making in Manufacturing and Services*, 8(1-2):13–23, 2014.
- 1730 [43] Benjamin Blonder, Christine Lamanna, Cyrille Violle, and Brian J Enquist. The n-dimensional
1731 hypervolume. *Global Ecology and Biogeography*, 23(5):595–609, 2014.
- 1732 [44] Lyndon While, Lucas Bradstreet, and Luigi Barone. A fast way of calculating exact hypervol-
1733 umes. *IEEE Transactions on Evolutionary Computation*, 16(1):86–95, 2011.
- 1734 [45] Andrzej Jaskiewicz and Piotr Zielniewicz. Quick extreme hypervolume contribution algorithm.
1735 In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 412–420,
1736 2021.
- 1737 [46] Peter I Frazier. A tutorial on bayesian optimization. *arXiv preprint arXiv:1807.02811*, 2018.
- 1738 [47] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna:
1739 A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM*
1740 *SIGKDD international conference on knowledge discovery & data mining*, pages 2623–2631,
1741 2019.
- 1742 [48] László Erdős, Antti Knowles, Horng-Tzer Yau, and Jun Yin. Spectral statistics of erdős-rényi
1743 graphs i: Local semicircle law. 2013.
- 1744 [49] Siddharth Pal and Ananthram Swami. Geometric nodal degree distributions arise in barabási-
1745 albert graphs! *IEEE Transactions on Network Science and Engineering*, 9(3):1409–1421,
1746 2022.